UNIVERSIDADE SÃO JUDAS TADEU — USJT
ÁREA DE CIÊNCIAS EXATAS, TECNOLOGIA E ENGENHARIA
UNIDADE CURRICULAR: SISTEMAS DISTRIBUÍDOS, INTERNET DAS COISAS E ENGENHARIA DE SOFTWARE (PROJETO A3)

EQUIPE DE ENGENHARIA DO PROJETO A3
Alexandre Coutinho Mendes – RA 826126504
Asafe Alves Octaviani – RA 825157068
Camilly Vitória da Silva Leal Campos – RA 825169119
Daniel Ribeiro Antunes – RA 826191361
Gustavo Leite de Oliveira – RA 825233821
Gustavo Sales da Silva – RA 826194498
Heloisa Soares Gomes – RA 8261108500
Renata Beatriz da Silva Carnajal – RA 825248240

CURRAL INTELIGENTE COM HIGIENIZAÇÃO AUTÔNOMA E PROTEÇÃO ANIMAL (AGRO 4.0):

ARQUITETURA DISTRIBUÍDA ORIENTADA A EVENTOS COM ESP32, MENSAGERIA MQTT, SPRING BOOT E STREAMING REATIVO VIA *SERVER-SENT EVENTS*

SÃO PAULO
2026

EQUIPE DE ENGENHARIA DO PROJETO A3
Alexandre Coutinho Mendes – RA 826126504
Asafe Alves Octaviani – RA 825157068
Camilly Vitória da Silva Leal Campos – RA 825169119
Daniel Ribeiro Antunes – RA 826191361
Gustavo Leite de Oliveira – RA 825233821
Gustavo Sales da Silva – RA 826194498
Heloisa Soares Gomes – RA 8261108500
Renata Beatriz da Silva Carnajal – RA 825248240

CURRAL INTELIGENTE COM HIGIENIZAÇÃO AUTÔNOMA E PROTEÇÃO ANIMAL (AGRO 4.0):

ARQUITETURA DISTRIBUÍDA ORIENTADA A EVENTOS COM ESP32, MENSAGERIA MQTT, SPRING BOOT E STREAMING REATIVO VIA *SERVER-SENT EVENTS*

Natureza do Trabalho (ABNT NBR 10719:2015 / NBR 14724:2011):
Relatório Técnico-Científico apresentado à Banca Examinadora da Unidade Curricular Projeto A3 da Universidade São Judas Tadeu (USJT), como requisito parcial para avaliação de desempenho acadêmico nas áreas de Sistemas Distribuídos, Internet das Coisas (IoT), Banco de Dados Relacional e Engenharia de Software.
Professor Responsável / Orientador: Prof. André Ferrus Filho
Código do Repositório: `com.usjt:a3` (`0.0.1-SNAPSHOT`)

SÃO PAULO
2026

RESUMO

A pecuária intensiva e semi-intensiva enfrenta desafios sanitários críticos decorrentes do acúmulo de dejetos orgânicos e da volatilização de gás amônia (NH_3) em currais de espera e ordenha, condição que eleva a incidência de mastite bovina, afecções podais e estresse respiratório no rebanho. Em contrapartida, a higienização manual ou automatizada com jatos hidráulicos de alta pressão sem controle rigoroso de presença animal representa grave risco de acidentes físicos e estresse agudo aos bovinos. Diante desse cenário, este relatório técnico apresenta o projeto, a implementação e a validação experimental do sistema **Curral Inteligente**, uma plataforma distribuída de Agropecuária de Precisão (Agro 4.0) concebida para automatizar a higienização de currais garantindo proteção biológica inviolável. A metodologia adotada fundamenta-se em uma arquitetura distribuída em quatro camadas desacopladas: (i) camada física embarcada em microcontrolador ESP32 DevKit C v4 (programado em C++ com suporte a duas variantes de hardware: versão híbrida com leitor RFID MFRC522 via barramento SPI e sensor de movimento PIR no pino GPIO 27, e versão enxuta exclusiva por sensor PIR, ambas com sensor de gases MQ-135 no conversor analógico-digital de 12 bits GPIO 34, sensor climático DHT22/DHT11 no GPIO 32 e módulo relé optoacoplado *Active-LOW* no GPIO 26 controlado por temporização não-bloqueante via `millis()`); (ii) camada de mensageria assíncrona baseada no protocolo MQTT v3.1.1 com *Quality of Service* nível 1 (QoS 1), endereçamento dinâmico *plug-and-play* pelo endereço MAC físico do chip e detecção de queda via *Last Will and Testament* (LWT); (iii) camada de backend desenvolvida em Java 25 e Spring Boot 4.2.0-M2, estruturada com roteamento de mensagens em tempo constante O(1) pelo padrão *Strategy*, isolamento *multi-tenant* estrito via tokens JWT *stateless*, tratamento global de exceções padronizado pela RFC 7807 (`ProblemDetail`) e três travas atômicas de segurança (*fail-fast safety interlocks*: trava biológica de presença animal, trava de concorrência física apoiada em índice parcial único no banco e trava de *heartbeat* com janela de 60 segundos), além de sincronização assíncrona via `CompletableFuture` sem retenção de conexões do pool HikariCP; e (iv) camada de persistência em PostgreSQL 17 versionada com Flyway 12.11.0, integrada a um frontend reativo em React 19.1 e TypeScript 5.9 alimentado em tempo real via *Server-Sent Events* (SSE) com *backoff* exponencial e parada biológica de emergência em camada dupla. A verificação empírica do sistema foi conduzida por meio de uma suíte exaustiva de 690 testes automatizados (345 testes no backend com JUnit 5, MockMvc e PostgreSQL real, e 345 testes no frontend com Vitest e React Testing Library), atingindo 100% de aprovação nos testes unitários, de componentes, de fronteira e adversariais, comprovando a robustez, a segurança biológica e a viabilidade operacional da solução.

Palavras-chave: Agropecuária de Precisão; Internet das Coisas; Sistemas Distribuídos; Arquitetura Orientada a Eventos; Bem-Estar Animal; Spring Boot.

ABSTRACT

Intensive and semi-intensive livestock farming faces critical sanitary challenges arising from the accumulation of organic waste and ammonia gas (NH_3) volatilization in holding and milking corrals, increasing the incidence of bovine mastitis, hoof disorders, and respiratory stress. Conversely, manual or automated high-pressure water cleaning without strict animal presence control poses severe risks of physical injury and acute stress to cattle. This technical report presents the design, implementation, and experimental validation of the **Smart Corral (Curral Inteligente)** system, a Precision Livestock Farming (Agro 4.0) distributed platform engineered to automate corral sanitization while enforcing inviolable biological protection. The methodology relies on a four-tier decoupled distributed architecture: (i) an embedded edge tier running on an ESP32 DevKit C v4 microcontroller (featuring two firmware variants: a hybrid MFRC522 SPI RFID and GPIO 27 PIR motion sensor version, and a lightweight PIR-only version, both equipped with an MQ-135 gas sensor on 12-bit ADC GPIO 34, a DHT22/DHT11 climate sensor on GPIO 32, and an *Active-LOW* optocoupled relay module on GPIO 26 driven by non-blocking `millis()` timing); (ii) an asynchronous messaging tier using MQTT v3.1.1 with QoS 1, dynamic *plug-and-play* MAC address topic routing, and *Last Will and Testament* (LWT) fault detection; (iii) a backend tier built with Java 25 and Spring Boot 4.2.0-M2, featuring O(1) *Strategy* pattern message routing, strict *multi-tenant* isolation via stateless JWT, RFC 7807 (`ProblemDetail`) standardized error handling, and three atomic *fail-fast* safety interlocks (biological presence lock, physical concurrency lock backed by a partial unique index, and a 60-second heartbeat lock), coupled with non-blocking `CompletableFuture` acknowledgment synchronization that prevents HikariCP connection pool starvation; and (iv) a relational persistence tier in PostgreSQL 17 managed by Flyway 12.11.0, paired with a reactive React 19.1 and TypeScript 5.9 frontend updated in real time via *Server-Sent Events* (SSE) with exponential backoff and dual-layer emergency biological stop. Empirical verification was conducted through an exhaustive suite of 690 automated tests (345 backend tests using JUnit 5 and 345 frontend tests using Vitest and React Testing Library), achieving a 100% pass rate across unit, component, boundary, and adversarial test suites, demonstrating the reliability, biological safety, and operational readiness of the proposed system.

Keywords: Precision Livestock Farming; Internet of Things; Distributed Systems; Event-Driven Architecture; Animal Welfare; Spring Boot.

LISTA DE ILUSTRAÇÕES

- **Figura 1** — Diagrama de Contexto e Arquitetura Geral em Quatro Camadas do Sistema Curral Inteligente
- **Figura 2** — Diagrama de Topologia de Hardware e Pinagem do Nó de Borda ESP32 (Versões V1 Híbrida e V2 PIR Active-LOW)
- **Figura 3** — Fluxograma Algorítmico das Três Travas Críticas de Segurança (*Fail-Fast Interlocks*) no `CleanService`
- **Figura 4** — Diagrama de Sequência UML do Ciclo de Acionamento da Bomba, Sincronização `AckService` e Parada de Emergência (`stopClean`)
- **Figura 5** — Diagrama de Sequência UML da Ingestão RFID, Regra de Fonte Única da Verdade (SSOT) e Triagem de Tags Desconhecidas (`IdentifyPendency`)
- **Figura 6** — Diagrama Entidade-Relacionamento (DER Físico) do Banco de Dados PostgreSQL 17 Gerenciado pelo Flyway 12
- **Figura 7** — Diagrama de Classes UML da Camada de Mensageria MQTT (Padrão *Strategy* O(1) e *Dependency Inversion*)
- **Figura 8** — Diagrama de Máquina de Estados Finitos (FSM) do Comando de Higienização (`CleanState`) e Conectividade (`DeviceStatus`)
- **Figura 9** — Diagrama de Componentes da Arquitetura Reativa do Frontend (React 19, Context API e *Server-Sent Events*)

LISTA DE QUADROS E TABELAS

- **Quadro 1** — Pilha Tecnológica (*Stack*) Consolidada por Camada Arquitetural
- **Quadro 2** — Mapeamento de Pinos (Pinout) e Periféricos do Microcontrolador ESP32 DevKit C v4
- **Quadro 3** — Comparativo Técnico entre as Variantes de Firmware Embarcado (`Iot/Iot.ino` vs. `IotV2/IotV2.ino`)
- **Quadro 4** — Topologia de Tópicos MQTT, Direção de Fluxo, QoS e Estrutura de Payloads JSON
- **Quadro 5** — Especificação das Três Travas de Segurança *Fail-Fast* e Mecanismo de Parada Imediata
- **Quadro 6** — Mapeamento de Exceções de Domínio para Respostas Padronizadas RFC 7807 (`ProblemDetail`)
- **Quadro 7** — Dicionário de Tabelas, Restrições de Integridade (`CHECK`) e Índices Parciais no PostgreSQL 17
- **Tabela 1** — Consolidação Quantitativa da Suíte de Testes Automatizados do Backend (JUnit 5 / Surefire)
- **Tabela 2** — Consolidação Quantitativa da Suíte de Testes Automatizados do Frontend (Vitest / React Testing Library)
- **Quadro 8** — Catálogo Completo de Endpoints da API REST (`/api/v1`)

LISTA DE ABREVIATURAS E SIGLAS

- **ABNT** — Associação Brasileira de Normas Técnicas
- **ACK** — *Acknowledgment* (Confirmação de Recebimento)
- **ADC** — *Analog-to-Digital Converter* (Conversor Analógico-Digital)
- **API** — *Application Programming Interface* (Interface de Programação de Aplicações)
- **BCrypt** — Algoritmo de *Hashing* Criptográfico Baseado no Cifrador Blowfish
- **CRUD** — *Create, Read, Update, Delete* (Operações Básicas de Persistência)
- **DDL** — *Data Definition Language* (Linguagem de Definição de Dados)
- **DER** — Diagrama Entidade-Relacionamento
- **DIP** — *Dependency Inversion Principle* (Princípio da Inversão de Dependência)
- **DTO** — *Data Transfer Object* (Objeto de Transferência de Dados)
- **EDA** — *Event-Driven Architecture* (Arquitetura Orientada a Eventos)
- **EMI** — *Electromagnetic Interference* (Interferência Eletromagnética)
- **FSM** — *Finite State Machine* (Máquina de Estados Finitos)
- **GPIO** — *General Purpose Input/Output* (Entrada/Saída de Propósito Geral)
- **HMAC** — *Hash-based Message Authentication Code*
- **HTTP** — *Hypertext Transfer Protocol*
- **IBGE** — Instituto Brasileiro de Geografia e Estatística
- **IDOR** — *Insecure Direct Object Reference* (Referência Direta Insegura a Objetos)
- **IoC** — *Inversion of Control* (Inversão de Controle)
- **IoT** — *Internet of Things* (Internet das Coisas)
- **JPA** — *Jakarta Persistence API*
- **JSON** — *JavaScript Object Notation*
- **JWT** — *JSON Web Token* (RFC 7519)
- **KPI** — *Key Performance Indicator* (Indicador-Chave de Desempenho)
- **LWT** — *Last Will and Testament* (Último Desejo e Testamento MQTT)
- **MAC** — *Media Access Control* (Controle de Acesso ao Meio)
- **MCU** — *Microcontroller Unit* (Unidade Microcontroladora)
- **MQTT** — *Message Queuing Telemetry Transport* (ISO/IEC 20922)
- **NBR** — Norma Brasileira
- **ORM** — *Object-Relational Mapping* (Mapeamento Objeto-Relacional)
- **PIR** — *Passive Infrared* (Sensor Infravermelho Passivo)
- **PPM** — Partes por Milhão
- **QoS** — *Quality of Service* (Qualidade de Serviço)
- **REST** — *Representational State Transfer*
- **RFC** — *Request for Comments* (Documento de Padronização da IETF)
- **RFID** — *Radio-Frequency Identification* (Identificação por Radiofrequência)
- **RTL** — *React Testing Library*
- **SGBD** — Sistema Gerenciador de Banco de Dados
- **SPA** — *Single-Page Application* (Aplicação de Página Única)
- **SPI** — *Serial Peripheral Interface*
- **SQL** — *Structured Query Language*
- **SSE** — *Server-Sent Events* (Eventos Enviados pelo Servidor — Padrão W3C)
- **SSOT** — *Single Source of Truth* (Fonte Única da Verdade)
- **UID** — *Unique Identifier* (Identificador Único)
- **UML** — *Unified Modeling Language* (Linguagem de Modelagem Unificada)
- **USJT** — Universidade São Judas Tadeu
- **UUID** — *Universally Unique Identifier*

SUMÁRIO

- **1 INTRODUÇÃO**
  - 1.1 Contextualização e Formulação do Problema
  - 1.2 Objetivos
    - 1.2.1 Objetivo Geral
    - 1.2.2 Objetivos Específicos
  - 1.3 Justificativa e Relevância Tecnológica
  - 1.4 Metodologia de Engenharia e Estrutura do Documento
- **2 FUNDAMENTAÇÃO TEÓRICA E ARQUITETURA DO SISTEMA**
  - 2.1 Agropecuária de Precisão (Agro 4.0) e Bem-Estar Animal
  - 2.2 Sistemas Distribuídos Orientados a Eventos, MQTT e *Server-Sent Events*
  - 2.3 Topologia Arquitetural em Quatro Camadas
  - 2.4 Pilha Tecnológica (*Stack*) Consolidada
- **3 ENGENHARIA DO SISTEMA EMBARCADO E BORDA IOT (ESP32)**
  - 3.1 Topologia de Hardware, Pinagem e Variantes de Firmware (V1 e V2)
  - 3.2 Lógica Elétrica do Módulo Relé *Active-LOW* e Prevenção de *Glitch* de Inicialização
  - 3.3 Temporização Não-Bloqueante via `millis()` e Autonomia Operacional na Borda
  - 3.4 Identificação Dinâmica *Plug-and-Play* por Endereço MAC e Tópicos MQTT
  - 3.5 Telemetria Ambiental (MQ-135 e DHT22/DHT11) e *Last Will and Testament* (LWT)
- **4 ENGENHARIA DO BACKEND E MODELAGEM DE DADOS**
  - 4.1 Padrões de Projeto Aplicados: *Strategy* O(1) e Inversão de Dependência (DIP)
  - 4.2 As Três Travas Críticas de Segurança (*Fail-Fast*) e Parada Imediata (`stopClean`)
  - 4.3 Sincronização Assíncrona (`AckService`) e Prevenção de *HikariCP Connection Starvation*
  - 4.4 Fonte Única da Verdade (SSOT) em Eventos RFID e Triagem de Tags Desconhecidas (`IdentifyPendency`)
  - 4.5 Isolamento *Multi-Tenant*, Autenticação JWT Híbrida e Tratamento de Erros RFC 7807
  - 4.6 Modelagem Relacional no PostgreSQL 17, Migrações Flyway e Índices Parciais
- **5 ENGENHARIA DO FRONTEND REATIVO E STREAMING EM TEMPO REAL**
  - 5.1 Arquitetura Modular em React 19 e Gerenciamento de Contexto Global
  - 5.2 Streaming Reativo via *Server-Sent Events* com *Backoff* Exponencial e *Fallback* de *Polling*
  - 5.3 Parada Biológica de Emergência em Camada Dupla (*Defense in Depth*) e Modo Simulado
- **6 VERIFICAÇÃO, VALIDAÇÃO E RESULTADOS EXPERIMENTAIS**
  - 6.1 Metodologia de Verificação e Cobertura de Testes Automatizados (690 Testes)
  - 6.2 Análise da Suíte de Testes do Backend (345 Testes em JUnit 5)
  - 6.3 Análise da Suíte de Testes do Frontend (345 Testes em Vitest e RTL)
- **7 CONCLUSÃO E TRABALHOS FUTUROS**
  - 7.1 Considerações Finais
  - 7.2 Limitações Identificadas e Direções para Trabalhos Futuros
- **REFERÊNCIAS**
- **APÊNDICE A — CATÁLOGO COMPLETO DE ENDPOINTS DA API REST**
- **APÊNDICE B — ESQUEMA DDL CONSOLIDADO DO BANCO DE DADOS (FLYWAY)**

1 INTRODUÇÃO

1.1 Contextualização e Formulação do Problema

A modernização das cadeias produtivas do agronegócio brasileiro, impulsionada pelo paradigma da Agropecuária de Precisão (Agro 4.0), demanda a integração convergente entre sensores físicos de borda, redes de telemetria sem fio e sistemas distribuídos em nuvem (Kleinschmidt *et al.*, 2019). Na bovinocultura leiteira e de corte em regime de confinamento ou semiconfinamento, os currais de manejo e salas de espera constituem pontos críticos de concentração biológica. O acúmulo contínuo de dejetos fecais e urina nesses recintos promove a degradação ureica e a rápida volatilização de gás amônia (NH_3) e sulfeto de hidrogênio (H_2S), deteriorando a qualidade do ar e favorecendo a proliferação de patógenos ambientais causadores de mastite clínica, dermatite digital e enfermidades respiratórias no rebanho.

Todavia, a automação da lavagem hidráulica de currais apresenta um dilema de engenharia de segurança: o acionamento de bombas d'água de alta pressão enquanto animais encontram-se no interior do recinto ocasiona estresse agudo, escorregões em piso úmido, traumas físicos e aversão comportamental ao manejo. Adicionalmente, sistemas convencionais baseados em temporizadores estáticos (relógios eletromecânicos) carecem de percepção contextual — lavando recintos ocupados ou desperdiçando recursos hídricos em currais limpos — e não oferecem rastreabilidade multi-fazenda ou garantias de concorrência quando múltiplos operadores interagem simultaneamente com a planta.

O problema de pesquisa e engenharia endereçado neste trabalho consiste em responder à seguinte questão técnica: **como projetar e implementar uma plataforma distribuída de higienização autônoma de currais que combine telemetria de gases, identificação automatizada de animais e atuação remota não-bloqueante, assegurando, de forma determinística e tolerante a falhas, que nenhuma bomba hidráulica opere na presença de animais ou sob perda de conectividade de hardware?**

1.2 Objetivos

1.2.1 Objetivo Geral

Projetar, desenvolver e validar experimentalmente o sistema **Curral Inteligente**, uma plataforma distribuída de ponta a ponta (borda IoT em ESP32, mensageria MQTT, backend corporativo em Java 25 com Spring Boot 4.2 e frontend reativo em React 19) voltada à automação da higienização de currais bovinos com priorização estrita do bem-estar animal mediante travas de segurança *fail-fast* e monitoramento em tempo real.

1.2.2 Objetivos Específicos

1. Desenvolver o firmware embarcado em C++/Arduino para o microcontrolador ESP32 DevKit C v4 em duas variantes operacionais (`Iot/Iot.ino` híbrida com RFID MFRC522 + PIR e `IotV2/IotV2.ino` dedicada com sensor PIR), implementando chaveamento seguro *Active-LOW* do módulo relé no pino GPIO 26, temporização local não-bloqueante via `millis()` e endereçamento dinâmico *plug-and-play* baseado no endereço MAC físico do dispositivo;
2. Estruturar uma topologia de comunicação assíncrona sobre o protocolo MQTT v3.1.1 com garantia de entrega *Quality of Service* nível 1 (QoS 1), retenção de estado de conectividade e detecção automática de desconexão abrupta via *Last Will and Testament* (LWT);
3. Construir o núcleo de backend distribuído utilizando Java 25 e Spring Boot 4.2.0-M2, aplicando o padrão de projeto *Strategy* para roteamento de eventos MQTT em tempo constante O(1) e o Princípio da Inversão de Dependência (DIP) para desacoplar as regras de domínio da infraestrutura de transporte;
4. Implementar três travas críticas de segurança *fail-fast* no serviço de higienização (`CleanService`) — Trava Biológica, Trava de Concorrência Física e Trava de Conectividade/*Heartbeat* de 60 segundos — aliadas a um mecanismo de confirmação assíncrona (`AckService`) com `CompletableFuture` que elimine a retenção de conexões do banco de dados (*Zero HikariCP Connection Starvation*);
5. Modelar e versionar o esquema relacional no SGBD PostgreSQL 17 por meio do Flyway 12.11.0, incorporando restrições de integridade (`CHECK`), índices parciais únicos para idempotência física (`uq_clean_cmd_active_per_corral`) e suporte a reconciliação transacional de tags RFID desconhecidas (`identify_pendency`);
6. Desenvolver uma interface web reativa em React 19.1 e TypeScript 5.9 integrada via *Server-Sent Events* (SSE) com reconexão resiliente por *backoff* exponencial, *fallback* automático de *polling* e parada biológica de emergência em camada dupla (*Defense in Depth*);
7. Submeter todo o ecossistema a uma bateria rigorosa de testes automatizados unitários, de integração, de fronteira e adversariais (totalizando 690 testes automatizados entre backend e frontend).

1.3 Justificativa e Relevância Tecnológica

A relevância deste projeto manifesta-se em três dimensões complementares: sanitária e zootécnica, ambiental e de engenharia de sistemas distribuídos. Sob a ótica zootécnica, a remoção automatizada de dejetos condicionada à concentração de amônia aferida pelo sensor MQ-135 reduz a exposição do epitélio respiratório e da glândula mamária dos bovinos a agentes infecciosos, enquanto as travas biológicas

impedem acidentes hidráulicos. Sob a ótica ambiental e econômica, a substituição de lavagens empíricas por ciclos parametrizados (limitados a até 60 segundos por acionamento e disparados apenas sob demanda real de odor ou agendamento programado) promove economia substancial de água e energia elétrica.

No âmbito da Engenharia de Software e dos Sistemas Distribuídos, o projeto soluciona problemas clássicos de sistemas ciberfísicos: (a) a discrepância de velocidade entre redes de sensores sem fio (sujeitas a latências de segundos e quedas de pacote) e pools de conexões de bancos relacionais (que exigem transações de milissegundos); (b) a prevenção de condições de corrida (*race conditions*) em acionamentos simultâneos de atuadores físicos; e (c) a manutenção de uma Fonte Única da Verdade (*Single Source of Truth* — SSOT) diante de leituras ruidosas ou duplicadas na borda IoT (Tanenbaum; Van Steen, 2017; Kleppmann, 2017).

1.4 Metodologia de Engenharia e Estrutura do Documento

O desenvolvimento seguiu uma abordagem iterativa incremental orientada a testes (*Test-Driven Quality Assurance*), combinando prototipação virtual de hardware no simulador Wokwi e validação em bancada física ESP32 com desenvolvimento de software baseado em princípios SOLID, *Clean Architecture* e especificações abertas da IETF (RFC 7519 para JWT e RFC 7807 para `ProblemDetail`).

Este relatório está organizado em sete seções, em conformidade com a ABNT NBR 10719:2015. A Seção 2 apresenta a fundamentação teórica e a visão arquitetural em quatro camadas. A Seção 3 detalha o projeto de hardware embarcado e as duas versões do firmware IoT. A Seção 4 aprofunda a engenharia do backend Spring Boot, os padrões de projeto, as travas de segurança e a modelagem de dados no PostgreSQL 17. A Seção 5 descreve a arquitetura reativa do frontend React 19 e o protocolo SSE. A Seção 6 documenta a metodologia de verificação empírica e os resultados dos 690 testes automatizados. Por fim, a Seção 7 sintetiza as conclusões e propõe extensões futuras.

2 FUNDAMENTAÇÃO TEÓRICA E ARQUITETURA DO SISTEMA

2.1 Agropecuária de Precisão (Agro 4.0) e Bem-Estar Animal

A Agropecuária de Precisão consiste na aplicação de tecnologias de sensoriamento, processamento distribuído e automação para monitorar variáveis ambientais e fisiológicas em nível individual ou de lote, otimizando a tomada de decisão no campo. Em instalações de confinamento e ordenha, o monitoramento contínuo de gases tóxicos — especialmente a amônia (NH_3), cuja concentração acima de 25 ppm já provoca irritação de mucosas em bovinos e trabalhadores rurais — aliado ao controle de temperatura e umidade relativa (Índice de Temperatura e Umidade — ITU), permite intervir preventivamente sobre a higiene ambiental.

Contudo, em sistemas ciberfísicos agropecuários (*Agricultural Cyber-Physical Systems*), o atuador mecânico interage diretamente com seres vivos de grande porte. Por conseguinte, a arquitetura de software deve ser projetada sob o paradigma *Safety-First* (Segurança Crítica em Primeiro Lugar), segundo o qual qualquer incerteza quanto à ausência de animais no recinto ou quanto à integridade da comunicação com o controlador de borda deve conduzir o sistema imediatamente a um estado seguro (*fail-safe* / *fail-fast*), mantendo a bomba d'água desenergizada (Sommerville, 2019).

2.2 Sistemas Distribuídos Orientados a Eventos, MQTT e *Server-Sent Events*

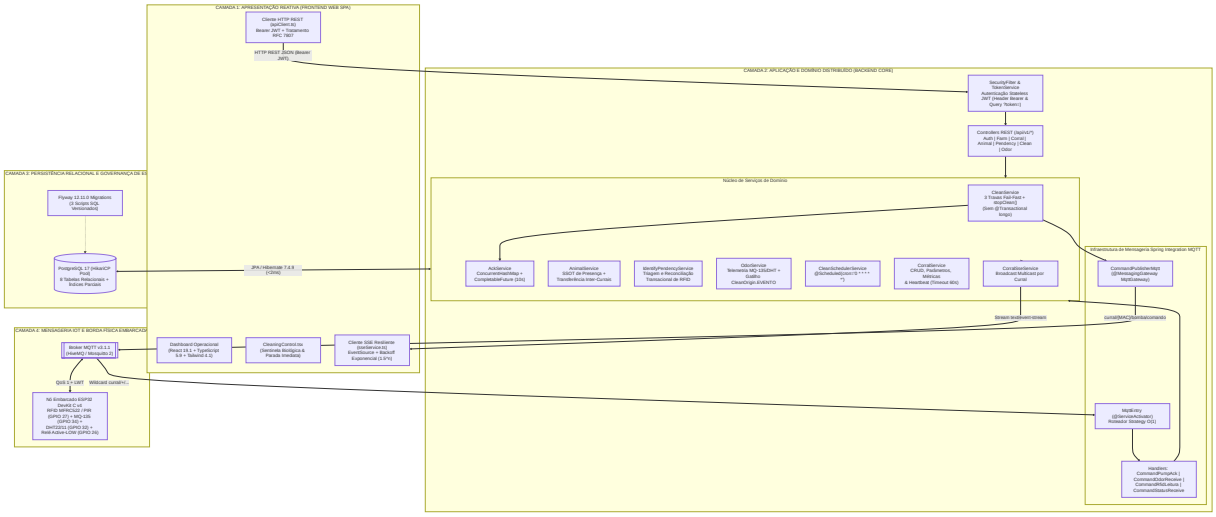
Em cenários rurais caracterizados por enlaces Wi-Fi sujeitos a atenuação e instabilidade, o acoplamento síncrono direto (como chamadas HTTP REST originadas pelo servidor em direção ao microcontrolador) mostra-se inadequado devido a bloqueios de *NAT/Firewall*, endereçamento IP dinâmico e alto *overhead* de cabeçalhos. Para superar essas limitações, o sistema adota uma Arquitetura Orientada a Eventos (*Event-Driven Architecture* — EDA) estruturada sobre dois protocolos complementares (Newman, 2021):

1. **MQTT v3.1.1 (ISO/IEC 20922) na Borda IoT:** Protocolo de publicação/assinatura (*publish/subscribe*) extremamente leve sobre TCP/IP, intermediado por um *broker* central (HiveMQ ou Eclipse Mosquitto). O uso de *Quality of Service* nível 1 (QoS 1 — *At least once delivery*) garante que comandos de acionamento da bomba e confirmações (*ACKs*) sejam retransmitidos até o recebimento do pacote `PUBACK`, enquanto o recurso de *Last Will and Testament* (LWT) delega ao *broker* a responsabilidade de notificar imediatamente o backend (`{"status":"OFFLINE"}`) caso o *keep-alive* do ESP32 expire sem fechamento gracioso de conexão;
2. ***Server-Sent Events* (SSE — Padrão W3C) na Interface Cliente:** Diferentemente do protocolo WebSocket — que estabelece um túnel TCP bidirecional complexo e frequentemente bloqueado por *proxies* corporativos —, o SSE opera sobre conexões HTTP padrão unidirecionais (`Content-Type: text/event-stream`), sendo ideal para o fluxo contínuo de telemetria e alertas do servidor para múltiplos navegadores de operadores rurais simultâneos.

2.3 Topologia Arquitetural em Quatro Camadas

A arquitetura global do sistema **Curral Inteligente** foi concebida em quatro camadas horizontais estritamente desacopladas, conforme ilustrado na Figura 1.

Figura 1 — Diagrama de Contexto e Arquitetura Geral em Quatro Camadas do Sistema Curral Inteligente



Fonte: Elaborado pelos autores (2026).

2.4 Pilha Tecnológica (*Stack*) Consolidada

O Quadro 1 sintetiza os componentes tecnológicos empregados em cada camada do projeto, extraídos diretamente dos descritores de construção `backend/pom.xml`, `frontend/package.json`, `backend/compose.yaml` e `Iot/libraries.txt`.

Quadro 1 — Pilha Tecnológica (*Stack*) Consolidada por Camada Arquitetural

Camada Arquitetural	Tecnologia / Biblioteca	Versão	Papel Técnico no Ecossistema
Camada 1: Frontend Reativo	React / React DOM	`19.1.0`	Construção da interface declarativa baseada em componentes e *Hooks*.
Camada 1: Frontend Reativo	TypeScript	`5.9.3`	Tipagem estática estrita de DTOs, contratos REST, eventos SSE e estados.
Camada 1: Frontend Reativo	Vite	`7.3.2`	Bundler ESM de alta velocidade e servidor de desenvolvimento.
Camada 1: Frontend Reativo	TailwindCSS	`4.1.14`	Estilização utilitária responsiva com suporte a temas e indicadores semânticos.
Camada 1: Frontend Reativo	Lucide React	`0.545.0`	Iconografia vetorial leve e acessível (`aria-hidden`).
Camada 1: Frontend (Testes)	Vitest + React Testing Library	`3.2.7` / `16.3.3`	Execução de 345 testes unitários, de componentes, de estresse e de integração.
Camada 2: Backend Distribuído	Java (Eclipse Temurin)	`25`	Plataforma de execução moderna com *Records*, *Pattern Matching* e *Switch Expressions*.
Camada 2: Backend Distribuído	Spring Boot	`4.2.0-M2`	Framework corporativo de Inversão de Controle (IoC), WebMVC, Validação e Agendamento.
Camada 2: Backend Distribuído	Spring Security + Auth0 JWT	`4.5.0`	Cadeia de filtros *stateless*, autenticação híbrida (Header/Query) e criptografia BCrypt.
Camada 2: Backend Distribuído	Spring Integration MQTT + Paho	`7.2.0` / `1.2.5`	Adaptadores de canal (`DirectChannel`), `@ServiceActivator` e `@MessagingGateway`.
Camada 2: Backend (Testes)	JUnit 5 + Spring Boot Test	`4.2.0-M2`	Execução de 345 testes automatizados (unitários, MockMvc, adversariais e banco real).
Camada 3: Banco de Dados	PostgreSQL	`17`	SGBD relacional transacional ACID com suporte a expressões regulares e índices parciais.
Camada 3: Banco de Dados	Flyway Database Migrations	`12.11.0`	Governança e versionamento determinístico do esquema relacional (`db/migration`).
Camada 3: Persistência ORM	Hibernate ORM / Spring Data JPA	`7.4.9`	Mapeamento objeto-relacional otimizado com `@EntityGraph` e *Derived Queries*.
Camada 4: Borda IoT & Broker	ESP32 DevKit C v4 (C++/Wokwi)	Arduino Core	Microcontrolador Wi-Fi dual-core executando os firmwares `Iot.ino` e `IotV2.ino`.
Camada 4: Borda IoT & Broker	PubSubClient / ArduinoJson / DHT	`2.8` / `6.x`	Cliente MQTT embarcado, serialização JSON em *stack* estática e leitura climática.
Camada 4: Borda IoT & Broker	HiveMQ / Eclipse Mosquitto	`2.0`	*Broker* MQTT v3.1.1 responsável pelo roteamento de tópicos com curinga (`+`) e LWT.

Fonte: Elaborado pelos autores com base nos arquivos de configuração do repositório (2026).

3 ENGENHARIA DO SISTEMA EMBARCADO E BORDA IOT (ESP32)

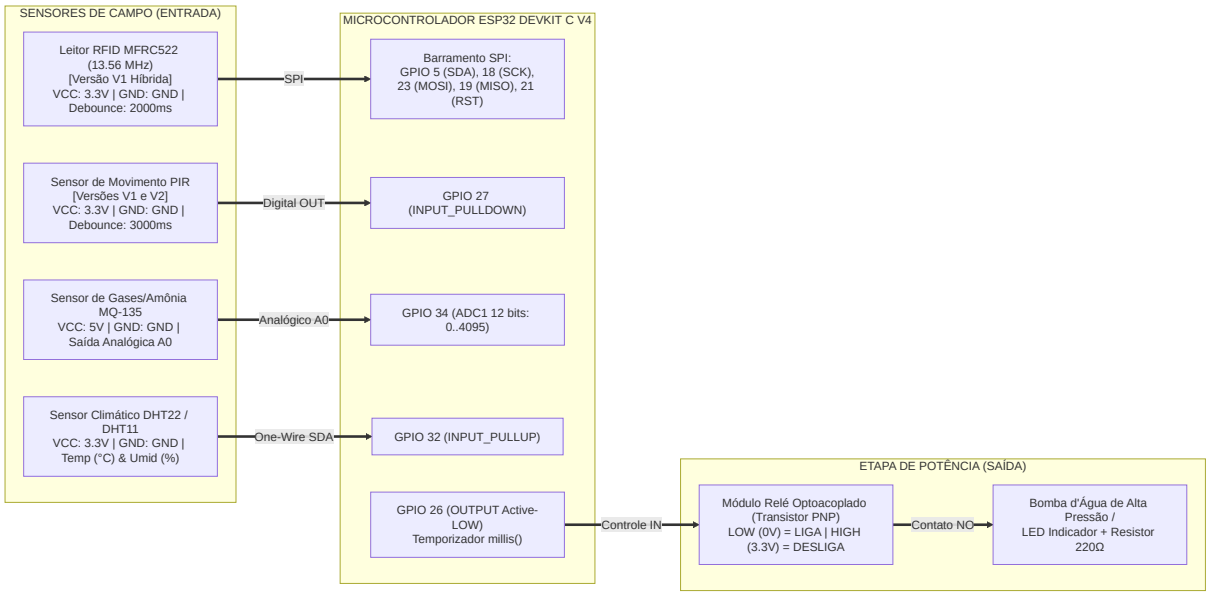
3.1 Topologia de Hardware, Pinagem e Variantes de Firmware (V1 e V2)

Cada curral monitorado é equipado com uma unidade microcontroladora **ESP32 DevKit C v4** (arquitetura 1:1 entre placa e curral). Durante o ciclo de engenharia do projeto, foram desenvolvidas e validadas duas versões de firmware e circuito eletrônico no repositório, atendendo tanto a cenários de simulação e identificação individual por rádio quanto a montagens físicas simplificadas em bancada:

1. **Versão V1 — Firmware Híbrido RFID + PIR (`Iot/Iot.ino` e `Iot/diagram.json`):** Combina simultaneamente o leitor de radiofrequência **MFRC522 (13,56 MHz)** conectado ao barramento SPI hardware do ESP32 (capaz de ler múltiplos brincos eletrônicos distintos e rastrear um vetor dinâmico `std::vector<String> tagsNoCurral` com *debounce* de 2.000 ms) e o sensor infravermelho passivo **PIR (GPIO 27)**;
2. **Versão V2 — Firmware Dedicado PIR & Relé *Active-LOW* (`IotV2/IotV2.ino` e `IotV2/diagram.json`):** Elimina as dependências das bibliotecas `SPI.h` e `MFRC522.h`, operando exclusivamente com o **Sensor de Movimento PIR no pino GPIO 27** (com *debounce* de 3.000 ms e detecção por borda de subida `LOW -> HIGH`). Na primeira passagem detectada pelo PIR, o firmware publica o evento `"ENTROU"` com uma tag RFID configurável (padrão `"E2801160"`, correspondente à Vaca Mimosa pré-cadastrada); na segunda passagem, publica `"SAIU"`, mantendo compatibilidade transparente de 100% com o contrato MQTT esperado pelo backend Spring Boot sem exigir alterações no servidor.

A Figura 2 apresenta o diagrama de blocos e conexões elétricas do nó embarcado, enquanto os Quadros 2 e 3 detalham, respectivamente, a tabela de pinagem (*pinout*) e o comparativo entre as versões V1 e V2.

Figura 2 — Diagrama de Topologia de Hardware e Pinagem do Nó de Borda ESP32 (Versões V1 Híbrida e V2 PIR Active-LOW)



Fonte: Elaborado pelos autores com base em `Iot/diagram.json` e `IotV2/diagram.json` (2026).

Quadro 2 — Mapeamento de Pinos (Pinout) e Periféricos do Microcontrolador ESP32 DevKit C v4

Componente Físico	Pino no ESP32	Configuração Elétrica no Firmware	Função Operacional no Curral Inteligente
Módulo Relé 5V (Bomba)	`GPIO 26`	`OUTPUT` (*Active-LOW*: `LOW` = Ligado, `HIGH` = Desligado)	Chaveamento eletromecânico da contatora da bomba d'água de lavagem.
Sensor de Movimento PIR	`GPIO 27`	`INPUT_PULLDOWN` (Borda de subida `LOW -> HIGH`)	Detecção física de passagem de animais com filtro de *debounce* de 3.000 ms.
Sensor Climático DHT22/11	`GPIO 32`	`INPUT_PULLUP` (`DHT22` em `Iot.ino` / `DHT11` em `IotV2.ino`)	Aferição periódica de temperatura ambiente (°C) e umidade relativa do ar (%).
Sensor de Gases MQ-135	`GPIO 34`	`INPUT` (Conversor Analógico-Digital ADC1 de 12 bits)	Leitura bruta (0 a 4095) da concentração de gases e amônia (NH_3) a cada 15 s.
Leitor RFID MFRC522	`GPIO 5` (`SDA/SS`), `GPIO 21` (`RST`), `GPIO 18` (`SCK`), `GPIO 19` (`MISO`), `GPIO 23` (`MOSI`)	Barramento `SPI` Hardware (Presente em `Iot/Iot.ino`)	Identificação individual de brincos eletrônicos passivos com *debounce* de 2.000 ms.

Fonte: Elaborado pelos autores com base em `Iot/Iot.ino` e `IotV2/IotV2.ino` (2026).

Quadro 3 — Comparativo Técnico entre as Variantes de Firmware Embarcado (`Iot/Iot.ino` vs. `IotV2/IotV2.ino`)

Critério Técnico	Versão V1 Híbrida (`Iot/Iot.ino`)	Versão V2 Dedicada PIR (`IotV2/IotV2.ino`)
Mecanismo de Detecção Animal	Híbrido: Leitor RFID MFRC522 (SPI) + Sensor PIR (GPIO 27) simultâneos.	Exclusivo por Sensor de Movimento PIR (GPIO 27), sem barramento SPI.
Rastreamento de Presença	Multi-animal real via `std::vector<String> tagsNoCurral` + alternância por UID.	Alternância binária (`1ª passagem = ENTROU`, `2ª passagem = SAIU`) com tag configurável.
Lógica de Acionamento do Relé	*Active-LOW* (`RELE_LIGADO = LOW`, `RELE_DESLIGADO = HIGH`) com *boot-safe*.	*Active-LOW* (`RELE_LIGADO = LOW`, `RELE_DESLIGADO = HIGH`) com *boot-safe* e envio prévio de ACK antes do chaveamento indutivo.
Suporte a Parada Remota (`PARAR`)	Sim (`acao == "PARAR"`, `"STOP"` ou `duracao <= 0`).	Sim (`acao == "PARAR"`, `"STOP"` ou `duracao <= 0`).
Telemetria Climática acoplada	Envia `valor`, `temperatura` e `umidade` no tópico `odor/nivel` (`TIPO_DHT = DHT22`).	Envia `valor`, `temperatura` e `umidade` no tópico `odor/nivel` (`TIPO_DHT = DHT11`).

Fonte: Elaborado pelos autores (2026).

3.2 Lógica Elétrica do Módulo Relé *Active-LOW* e Prevenção de *Glitch* de Inicialização

Um aspecto crítico de engenharia de hardware implementado em ambas as versões do firmware (`Iot.ino:38-39` e `IotV2.ino:48-49`) refere-se à polaridade elétrica dos módulos de relé optoacoplados industriais e comerciais. Nesses módulos, o LED interno do optoacoplador (ou a base do transistor PNP de acionamento, configurado no simulador Wokwi com o atributo `"transistor": "pnp"`) é energizado quando o pino de controle `IN` é drenado para o potencial de terra (`LOW` / 0 V), e desenergizado quando mantido em nível lógico alto (`HIGH` / 3,3 V).

Se um microcontrolador configurar o pino `GPIO 26` diretamente como saída (`pinMode(PIN_RELE, OUTPUT)`) antes de definir seu registrador de saída (`GPIO_OUT_REG`), o valor padrão zero do registrador faz com que o pino assuma nível `LOW` por alguns milissegundos durante o *boot*, provocando um acionamento involuntário (*glitch*) da bomba d'água sempre que a placa reinicia. Para eliminar esse risco físico, o procedimento de inicialização (`setup()`) grava explicitamente o estado `RELE_DESLIGADO` (`HIGH`) no registrador de dados **antes** de habilitar o buffer de saída do pino:

```
// Prevenção de glitch em periféricos Active-LOW durante o boot do ESP32:
digitalWrite(PIN_RELE, RELE_DESLIGADO); // Define HIGH no registrador antes de habilitar saída
pinMode(PIN_RELE, OUTPUT);              // Habilita o driver de saída já em estado seguro (OFF)
```

Adicionalmente, em `IotV2/IotV2.ino`, ao receber um comando de lavagem, o firmware transmite o pacote MQTT de `ACK` e executa `mqttClient.loop()` com uma breve estabilização de 50 ms **antes** de fechar o contato do relé (`ligarBomba(duracaoSegundos)`). Essa decisão de engenharia previne que quedas momentâneas de tensão na fonte de alimentação ou ruídos eletromagnéticos (EMI) gerados pela corrente de partida (*inrush current*) da bomba hidráulica derrubem o transceptor Wi-Fi antes que o backend receba a confirmação do comando.

3.3 Temporização Não-Bloqueante via `millis()` e Autonomia Operacional na Borda

Em sistemas embarcados de tempo real, o uso da primitiva bloqueante `delay(duracaoMs)` durante o acionamento de um atuador paralisa a CPU pelo período integral da lavagem (até 60 segundos). Caso isso ocorresse no Curral Inteligente, o ESP32 deixaria de processar o `mqttClient.loop()` (causando desconexão por *timeout* de *keep-alive* no broker), ignoraria comandos de parada de emergência (`"acao": "PARAR"`) e ficaria cego à entrada de um animal pelo sensor RFID ou PIR durante a lavagem.

Para garantir multitarefa cooperativa determinística, o controle da bomba foi projetado de forma 100% não-bloqueante utilizando o contador de ciclo de hardware `millis()` (`Iot.ino:363-366` e `IotV2.ino:386-389`):

```
// No loop() principal — executado milhares de vezes por segundo sem bloqueio:
if (bombaAtiva && (millis() - tempoInicioBomba >= duracaoBombaMs)) {
  desligarBomba();
  Serial.println("[HARDWARE] Duração da lavagem finalizada. Bomba d'água DESLIGADA!");
}
```

Essa arquitetura confere **autonomia operacional de segurança na borda**: a duração da lavagem é transmitida dentro do próprio payload do comando MQTT (`"duracao": 30`). Ainda que o cabo de internet da fazenda seja rompido ou o roteador Wi-Fi desligue um milissegundo após o acionamento da bomba, o microcontrolador ESP32 independe de uma segunda mensagem de rede para encerrar o ciclo, desligando o relé localmente com precisão de milissegundos assim que `millis() - tempoInicioBomba >= duracaoBombaMs`.

3.4 Identificação Dinâmica *Plug-and-Play* por Endereço MAC e Tópicos MQTT

Para viabilizar a produção em escala sem necessidade de compilar firmwares customizados com identificadores fixos (`corralId = 1`, `corralId = 2`), o nó embarcado extrai dinamicamente seu endereço físico de camada de enlace (`WiFi.macAddress()`) durante a conexão Wi-Fi, removendo separadores (`:` e `-`) e convertendo os 12 caracteres hexadecimais para caixa alta (ex.: `240AC4000110`). Todos os tópicos de publicação e assinatura são construídos em tempo de execução a partir do prefixo base `a3-usjt-fazenda-inteligente/curral/{MAC}/...`, conforme especificado no Quadro 4.

Quadro 4 — Topologia de Tópicos MQTT, Direção de Fluxo, QoS e Estrutura de Payloads JSON

Tópico MQTT (`prefixo = a3-usjt-fazenda-inteligente/curral`)	Direção do Fluxo	QoS / Retain	Exemplo de Payload JSON	Handler Responsável no Backend
`{prefixo}/{MAC}/bomba/comando`	Backend → ESP32	QoS 1 / `retain=false`	`{"comandoId":"8210ae63-...","duracao":30}` ou `{"comandoId":"...","acao":"PARAR","duracao":0}`	Publicado por `CommandPublisherMqt`
`{prefixo}/{MAC}/bomba/ack`	ESP32 → Backend	QoS 0 / `retain=false`	`{"comandoId":"8210ae63-...","status":"OK"}`	`CommandPumpAck`
`{prefixo}/{MAC}/rfid/leitura`	ESP32 → Backend	QoS 0 / `retain=false`	`{"tagUid":"E2801160","typeEvent":"ENTROU"}`	`CommandRfidLeitura`
`{prefixo}/{MAC}/odor/nivel`	ESP32 → Backend	QoS 0 / `retain=false`	`{"valor":412,"temperatura":27.4,"umidade":63.2}`	`CommandOdorReceive`
`{prefixo}/{MAC}/status`	ESP32 / Broker → Backend	QoS 1 / `retain=true`	`{"status":"ONLINE"}` (Conexão) / `{"status":"OFFLINE"}` (LWT)	`CommandStatusRecei`

Fonte: Elaborado pelos autores com base em `EntryTopics.java`, `MqttEntry.java` e `Iot.ino` (2026).

3.5 Telemetria Ambiental (MQ-135 e DHT22/DHT11) e *Last Will and Testament* (LWT)

A cada intervalo não-bloqueante de 15.000 ms (`INTERVALO_TELEMETRIA`), a função `enviarTelemetria()` realiza a leitura do conversor analógico-digital de 12 bits no pino `GPIO 34` (`analogRead(PIN_MQ135)`), restringindo o valor à faixa operacional segura (`constrain(leituraBruta, 0, 10000)`), e coleta simultaneamente a temperatura (°C) e a umidade relativa (%) através do sensor climático DHT no pino `GPIO 32` (tratando leituras inválidas `isnan()` com *fallback* seguro `0.0` e arredondamento de uma casa decimal). O pacote JSON consolidado `{"valor": 412, "temperatura": 27.4, "umidade": 63.2}` é publicado no tópico `{prefixo}/{MAC}/odor/nivel`, cumprindo duplo propósito no backend: registrar a série histórica ambiental e renovar passivamente o *heartbeat* (`last_seen_at`) do curral.

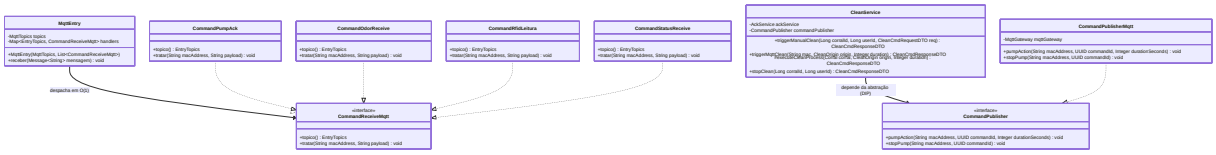
4 ENGENHARIA DO BACKEND E MODELAGEM DE DADOS

4.1 Padrões de Projeto Aplicados: *Strategy* O(1) e Inversão de Dependência (DIP)

O backend do sistema **Curral Inteligente** foi construído em **Java 25** sobre o framework **Spring Boot 4.2.0-M2**, adotando uma arquitetura em camadas limpa (*Controller → Service → Repository* / *Domain Ports ← MQTT Adapters*). Dois padrões de projeto (*Design Patterns*) estruturam o núcleo de integração IoT, conforme modelado na Figura 7:

1. **Padrão Comportamental *Strategy* com Despacho em O(1) (`MqttEntry`):** Um único adaptador `MqttPahoMessageDrivenChannelAdapter` subscreve os tópicos de entrada usando o caractere curinga de nível único (+). Ao receber uma mensagem no canal `mqttInputChannel`, o componente `@ServiceActivator MqttEntry` utiliza a classe `MqttTopics` para extrair o endereço MAC e o enum `EntryTopics`. Em vez de utilizar cadeias condicionais `if/else` ou `switch` que violariam o Princípio Aberto-Fechado (OCP), o construtor de `MqttEntry` recebe do contêiner IoC do Spring a lista polimórfica `List<CommandReceiveMqtt>` e indexa as estratégias em um `Map<EntryTopics, CommandReceiveMqtt>`. O roteamento ocorre em tempo constante O(1) via `handlers.get(req.tipo()).tratar(req.mac(), mensagem.getPayload())`;
2. **Princípio da Inversão de Dependência — DIP (`CommandPublisher`):** O serviço de domínio `CleanService` não conhece detalhes de protocolo MQTT, tópicos ou serialização JSON; ele depende exclusivamente da interface de domínio `CommandPublisher` (`pumpAction` e `stopPump`). A implementação concreta `CommandPublisherMqtt` reside no pacote de infraestrutura `com.usjt.a3.mqtt.exit`, permitindo que testes unitários e de fronteira substituam o transporte por *mocks* determinísticos sem acoplamento ao broker externo.

Figura 7 — Diagrama de Classes UML da Camada de Mensageria MQTT (Padrão *Strategy* O(1) e *Dependency Inversion*)



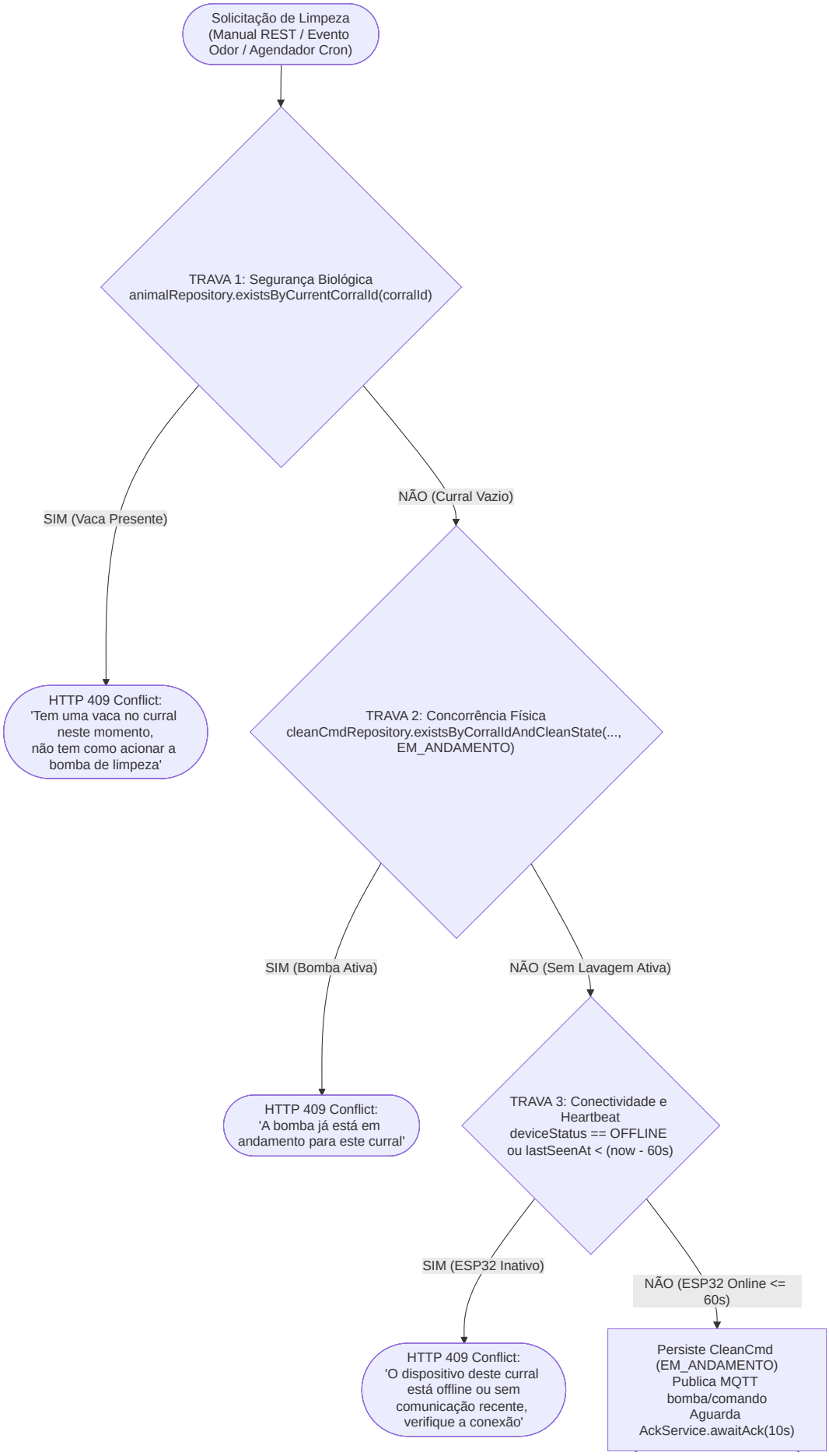
Fonte: Elaborado pelos autores com base nos pacotes `com.usjt.a3.mqtt` e `com.usjt.a3.service` (2026).

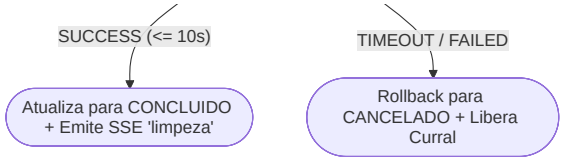
4.2 As Três Travas Críticas de Segurança (*Fail-Fast*) e Parada Imediata (`stopClean`)

O sistema suporta três origens de acionamento da higienização (`CleanOrigin`): **Manual** (via `POST /api/v1/currais/{corralId}/limpeza`), **Por Evento de Odor** (disparada autonomamente por `OdorService` quando a leitura do MQ-135 atinge `value >= corral.getLimitOdor()`) e **Automática Agendada** (disparada minuto a minuto por `CleanSchedulerService` via `@Scheduled(cron = "0 * * * * *")` quando o horário corrente coincide com `corral.getCleanTime()`).

Independentemente da origem do gatilho, todo fluxo converge obrigatoriamente para o método protegido `CleanService.executeCleanProcess`, no qual três travas de segurança atômicas e sequenciais (*fail-fast safety interlocks*) são avaliadas antes que qualquer pacote MQTT seja emitido, conforme detalhado na Figura 3 e no Quadro 5.

Figura 3 — Fluxograma Algorítmico das Três Travas Críticas de Segurança (*Fail-Fast Interlocks*) no `CleanService`


Solicitação de Limpeza
(Manual REST / Evento
Odor / Agendador Cron)
TRAVA 1: Segurança Biológica
animalRepository.existsByCurrentCorralId(corralId)
SIM (Vaca Presente)
NÃO (Curral Vazio)
HTTP 409 Conflict:
'Tem uma vaca no curral
neste momento,
não tem como acionar a
bomba de limpeza'
TRAVA 2: Concorrência Física
cleanCmdRepository.existsByCorralIdAndCleanState(...,
EM_ANDAMENTO)
SIM (Bomba Ativa)
NÃO (Sem Lavagem Ativa)
HTTP 409 Conflict:
'A bomba já está em
andamento para este curral'
TRAVA 3: Conectividade e
Heartbeat
deviceStatus == OFFLINE
ou lastSeenAt < (now - 60s)
SIM (ESP32 Inativo)
NÃO (ESP32 Online <=
60s)
HTTP 409 Conflict:
'O dispositivo deste curral
está offline ou sem
comunicação recente,
verifique a conexão'
Persiste CleanCmd
(EM_ANDAMENTO)
Publica MQTT
bomba/comando
Aguarda
AckService.awaitAck(10s)



Fonte: Elaborado pelos autores com base em `CleanService.java` (2026).

Quadro 5 — Especificação das Três Travas de Segurança *Fail-Fast* e Mecanismo de Parada Imediata

Mecanismo de Segurança	Localização no Código	Condição de Disparo / Verificação	Resposta / Comportamento de Segurança
Trava 1: Segurança Biológica	`CleanService.java:98-100`	`animalRepository.existsByCurrentCorralId(corralId) == true`	Aborta imediatamente com `IllegalStateException` (H `409 Conflict`). Protege os ani contra jatos d'água.
Trava 2: Concorrência Física	`CleanService.java:103-105` + `V202610152058:105`	`cleanCmdRepository.existsByCorralIdAndCleanState(corralId, EM_ANDAMENTO)` + índice `uq_clean_cmd_active_per_corral`	Aborta com HTTP `409 Confli` Impede sobrecarga hidráulica e condições de corrida entre operadores/cron.
Trava 3: Conectividade e *Heartbeat*	`CleanService.java:107-109`	`deviceStatus == OFFLINE` ou `lastSeenAt == null` ou `lastSeenAt < now().minusSeconds(60)`	Aborta com HTTP `409 Confli` Impede o envio de ordens de lava para placas desligadas ou sem red
Comando de Parada Imediata (`stopClean`)	`CleanService.java:156-202` + `V202610300101`	Chamada em `POST /api/v1/currais/{corralId}/limpeza/parar` (manual ou via sentinela biológica do frontend)	Cancela comandos `EM_ANDAMEN` banco, publica `{"acao":"PARAR","duracac` aguarda ACK e grava registro auc com `secondsDuration = 0`.

Fonte: Elaborado pelos autores (2026).

4.3 Sincronização Assíncrona (`AckService`) e Prevenção de *HikariCP Connection Starvation*

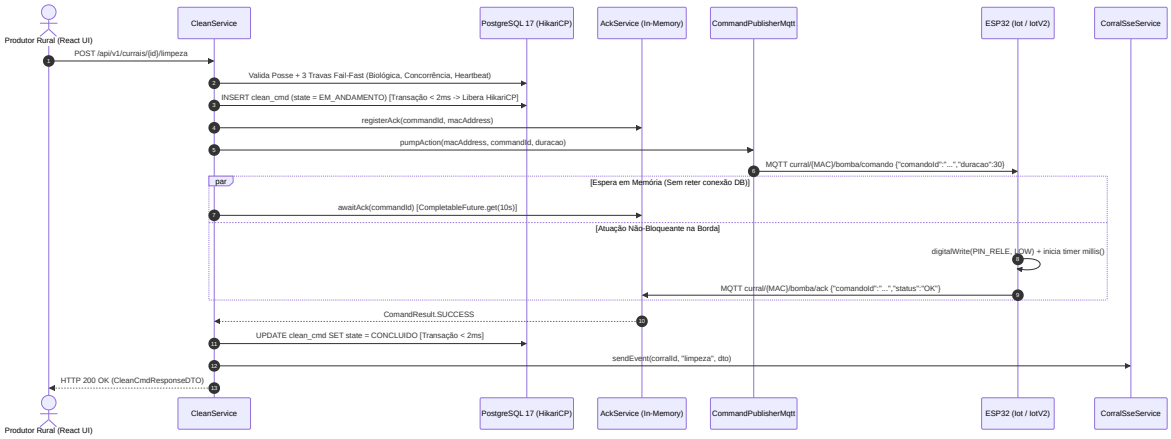
Uma decisão arquitetural de alta criticidade no método `CleanService.executeCleanProcess` consiste na **ausência deliberada da anotação `@Transactional` no método orquestrador**. Em aplicações Spring Boot convencionais, anotar um método de serviço com `@Transactional` faz com que o `DataSourceTransactionManager` aloque uma conexão física do pool **HikariCP** (cujo tamanho padrão é de apenas 10 conexões) no início do método e só a devolva ao final. Como o backend aguarda até **10 segundos** pela confirmação (`ACK`) do ESP32 via rede sem fio (`ackService.awaitAck(commandId)`), o uso de `@Transactional` causaria o esgotamento completo do pool de banco de dados (*Connection Pool Starvation*) caso apenas 10 currais iniciassem lavagens simultaneamente, paralisando toda a API para todos os produtores rurais.

Ao omitir `@Transactional` do método externo, o ciclo de vida é particionado em três fases não-bloqueantes para o banco de dados, conforme demonstrado na Figura 4:

1. **Transação Curta #1 (< 2 ms):** O comando `CleanCmd` é salvo com estado `EM_ANDAMENTO` via `cleanCmdRepository.save(cmd)`. O repositório Spring Data abre, comita a transação e **devolve imediatamente a conexão ao pool HikariCP**;
2. **Espera Assíncrona em Memória Sem Conexão de Banco (até 10 s):** O `AckService` mantém dois mapas concorrentes *thread-safe*: `ConcurrentHashMap<UUID, CompletableFuture<ComandResult>> pendingAck` e `ConcurrentHashMap<UUID, String> deviceAckResult`. A thread aguarda em `future.get(10000, TimeUnit.MILLISECONDS)`. Quando o ESP32 responde em `{prefixo}/{MAC}/bomba/ack`, o handler `CommandPumpAck` invoca `ackService.resolvedAck(uuid, macAddress, SUCCESS)`, validando criptograficamente se o endereço MAC que enviou o ACK é exatamente o mesmo MAC para o qual o comando foi emitido (prevenindo *spoofing* entre placas) e completando o `CompletableFuture` em O(1);
3. **Transação Curta #2 (< 2 ms) com Garantia de Desbloqueio:** Caso o resultado seja `ComandResult.SUCCESS`, o comando é atualizado para `CONCLUIDO`; caso ocorra `TIMEOUT` (> 10 s) ou falha de publicação MQTT (`FAILED`), o comando transiciona obrigatoriamente para `CANCELADO`, liberando o índice parcial único `uq_clean_cmd_active_per_corral` e garantindo que o curral jamais permaneça travado indefinidamente.

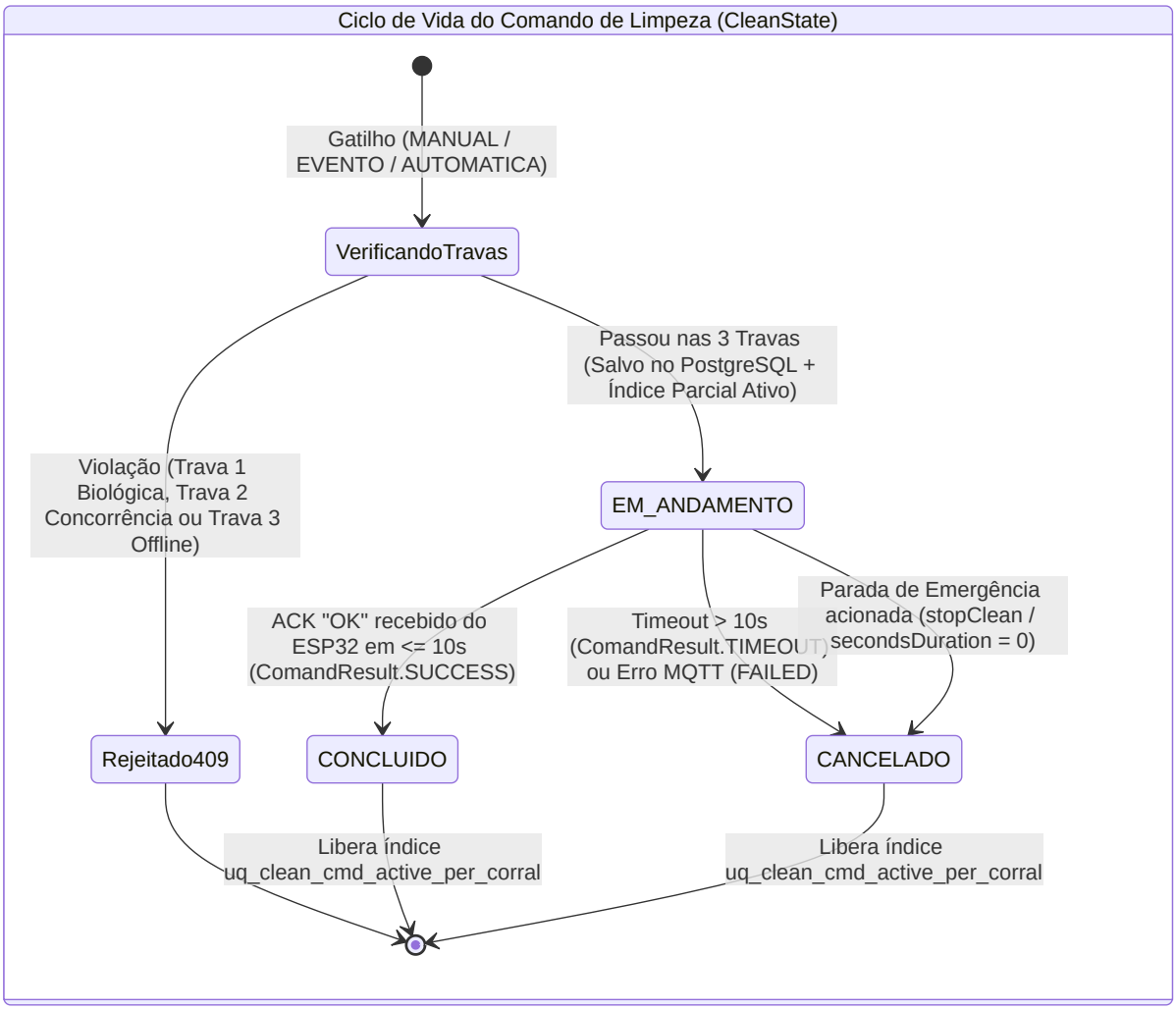
A Figura 4 ilustra o diagrama de sequência UML completo da sincronização assíncrona com o hardware, enquanto a Figura 8 formaliza a Máquina de Estados Finitos (FSM) tanto do comando de limpeza (`CleanState`) quanto do monitor de conectividade híbrida (`DeviceStatus`).

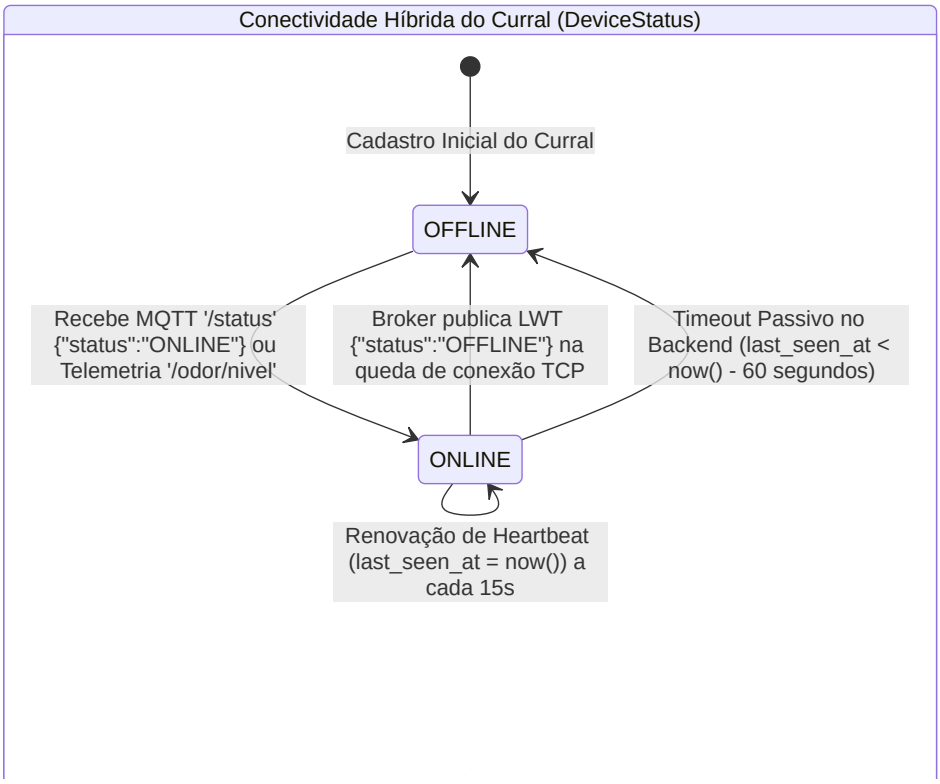
Figura 4 — Diagrama de Sequência UML do Ciclo de Acionamento da Bomba, Sincronização `AckService` e Parada de Emergência (`stopClean`)



Fonte: Elaborado pelos autores (2026).

Figura 8 — Diagrama de Máquina de Estados Finitos (FSM) do Comando de Higienização (`cleanState`) e Conectividade (`DeviceStatus`)


Ciclo de Vida do Comando de Limpeza (CleanState)
Gatilho (MANUAL / EVENTO / AUTOMATICA)
VerificandoTravas
Passou nas 3 Travas (Salvo no PostgreSQL + Índice Parcial Ativo)
Violação (Trava 1 Biológica, Trava 2 Concorrência ou Trava 3 Offline)
EM_ANDAMENTO
ACK "OK" recebido do ESP32 em <= 10s (ComandResult.SUCCESS)
Timeout > 10s (ComandResult.TIMEOUT) ou Erro MQTT (FAILED)
Parada de Emergência acionada (stopClean / secondsDuration = 0)
Rejeitado409
CONCLUIDO
CANCELADO
Libera índice uq_clean_cmd_active_per_corral
Libera índice uq_clean_cmd_active_per_corral


Conectividade Híbrida do Curral (DeviceStatus)
Cadastro Inicial do Curral
OFFLINE
Recebe MQTT '/status' {"status":"ONLINE"} ou Telemetria '/odor/nivel'
Broker publica LWT {"status":"OFFLINE"} na queda de conexão TCP
Timeout Passivo no Backend (last_seen_at < now() - 60 segundos)
ONLINE
Renovação de Heartbeat (last_seen_at = now()) a cada 15s

Fonte: Elaborado pelos autores com base em `CleanService`, `CorralService` e `CorralMapper` (2026).

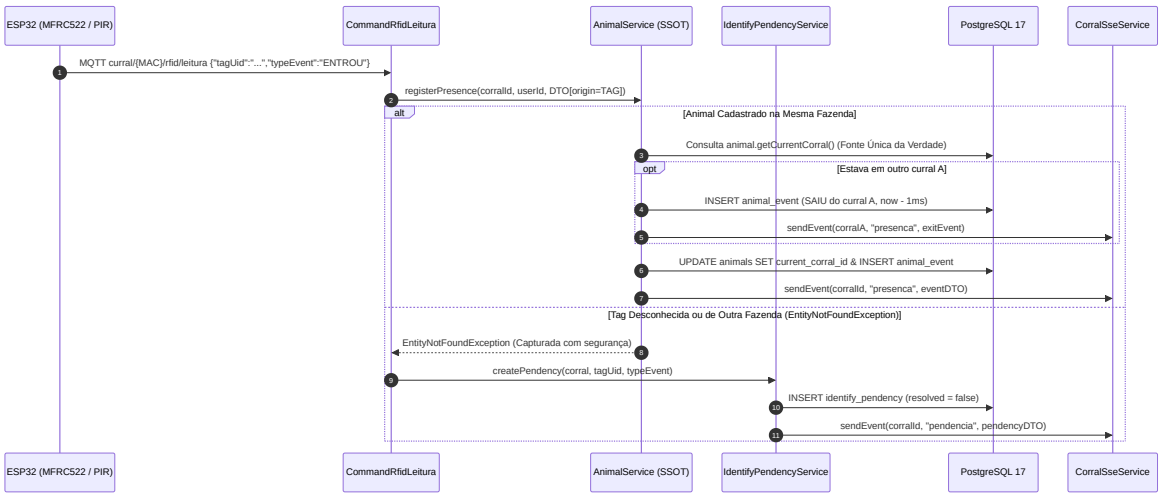
4.4 Fonte Única da Verdade (SSOT) em Eventos RFID e Triagem de Tags Desconhecidas (`IdentifyPendency`)

Na operação real de fazendas, leitores de radiofrequência e sensores ópticos na borda estão sujeitos a oscilações de energia (que limpam a memória RAM volátil do microcontrolador) ou múltiplas leituras de um mesmo brinco. Se o backend confiasse cegamente no campo `typeEvent` reportado pelo ESP32, um reinício da placa enquanto uma vaca estivesse dentro do curral faria com que sua saída física fosse reportada pela placa como uma segunda entrada (`ENTROU`), corrompendo a contagem de presença.

Para solucionar essa vulnerabilidade de consistência distribuída, o método `AnimalService.registerPresence` implementa o padrão **Fonte Única da Verdade (*Single Source of Truth* — SSOT)** para eventos originados por hardware (`EventOrigin.TAG`), conforme ilustrado na Figura 5:

1. **Determinação de Estado pelo Banco Relacional:** Quando `origin == EventOrigin.TAG`, o backend consulta o estado persistido de `animal.getCurrentCorral()` no PostgreSQL. Se o animal já consta como presente naquele curral (`currentCorral.getId().equals(corralId)`), o evento efetivo é forçado para `AnimalEventType.SAIU` e `currentCorral` é atualizado para `null`. Caso contrário, o evento efetivo é `AnimalEventType.ENTROU`;
2. **Transferência Atômica Inter-Currais:** Caso o animal estivesse alocado em um curral A e seja detectado entrando em um curral B da mesma fazenda (`currentCorral != null && !currentCorral.getId().equals(corralId)`), o serviço gera automaticamente um evento de saída (`SAIU`) no curral A com *timestamp* estritamente anterior (`LocalDateTime.now().minusNanos(1_000_000)`), emite a notificação SSE `"presenca"` para os observadores do curral A e, em seguida, registra a entrada (`ENTROU`) no curral B;
3. **Triagem Tolerante a Falhas de Tags Desconhecidas (`IdentifyPendencyService`):** Quando um brinco RFID não cadastrado na fazenda é lido pelo ESP32, `AnimalService` lança `EntityNotFoundException`. O handler `CommandRfidLeitura` intercepta essa exceção sem derrubar a thread do listener MQTT e invoca `identifyPendencyService.createPendency(corral, normalizedTagUid, typeEvent)`, gravando uma pendência aberta (`resolved = false`) e notificando imediatamente o painel do produtor via SSE (`"pendencia"`). Posteriormente, o produtor reconcilia a pendência em `POST /api/v1/pendencias/{pendencyId}/resolver`, transação que associa o animal, atualiza sua presença física e grava auditoria com `EventOrigin.MANUAL`.

Figura 5 — Diagrama de Sequência UML da Ingestão RFID, Regra de Fonte Única da Verdade (SSOT) e Triagem de Tags Desconhecidas (`IdentifyPendency`)



Fonte: Elaborado pelos autores (2026).

4.5 Isolamento *Multi-Tenant*, Autenticação JWT Híbrida e Tratamento de Erros RFC 7807

A segurança da API foi projetada sob arquitetura *Zero-Trust Multi-Tenant*, assegurando que produtores rurais distintos operem em ambientes logicamente isolados sobre a mesma infraestrutura compartilhada:

- **Autenticação JWT Híbrida para Compatibilidade W3C `EventSource` (`SecurityFilter`):** As senhas de usuários são armazenadas como *hashes* irreversíveis via `BCryptPasswordEncoder`. Na autenticação (`POST /api/v1/auth/login`), o `TokenService` emite um token JWT assinado com algoritmo `HMAC256`. Como a especificação nativa `EventSource` da W3C nos navegadores não suporta o envio de cabeçalhos customizados (`Authorization: Bearer`), o método `SecurityFilter.recoverToken(request)` implementa extração híbrida: inspeciona prioritariamente o cabeçalho HTTP `Authorization` e, caso ausente, extrai o parâmetro de consulta `?token=<jwt>` utilizado pelo canal de streaming `/api/v1/currais/{corralId}/stream`;

- **Prevenção de *Insecure Direct Object References* (IDOR):** Todas as operações de leitura e escrita validam a posse do ativo em relação ao `user.id()` extraído do `UserPrincipal` autenticado (`!farm.getUser().getId().equals(userId)`). Tentativas de acesso cruzado entre produtores disparam `SecurityException` convertida em HTTP `403 Forbidden`. Adicionalmente, rotas hierárquicas aninhadas (`/api/v1/farms/{farmId}/currais/{corralId}`) verificam a pertinência relacional entre a entidade filha e a fazenda informada na URL, retornando HTTP `404 Not Found` em caso de discrepância;
- **Travas de Exclusão Segura de Ciclo de Vida:** Os métodos `FarmService.deleteFarm` e `CorralService.deleteCorral` bloqueiam a remoção de fazendas ou currais que possuam comandos de limpeza em estado `EM_ANDAMENTO` ou `PENDENTE`, retornando HTTP `409 Conflict` e impedindo que bombas físicas fiquem órfãs de registro no banco;
- **Padronização Global de Erros RFC 7807 (`GlobalExceptionHandler`):** Todas as exceções lançadas pela aplicação são interceptadas por `@RestControllerAdvice` e serializadas na estrutura `ProblemDetail` (RFC 7807), conforme sintetizado no Quadro 6.

Quadro 6 — Mapeamento de Exceções de Domínio para Respostas Padronizadas RFC 7807 (`ProblemDetail`)

Exceção Interceptada no Backend	Status HTTP	Título (`title`) no `ProblemDetail`	Propriedades Adicionais / Casos de Uso
`MethodArgumentNotValidException`	`400 Bad Request`	`Erro de Validação`	Inclui mapa `invalidParams: Map<String, String>` por campo inválido.
`IllegalArgumentException` / `HttpMessageNotReadableException`	`400 Bad Request`	`Requisição Inválida` / `Corpo da Requisição Inválido`	Parâmetros ausentes, formatos inválidos ou JSON malformado.
`BadCredentialsException`	`401 Unauthorized`	`Credenciais Inválidas`	E-mail ou senha incorretos no fluxo de login.
`SecurityException`	`403 Forbidden`	`Acesso Negado`	Violação de isolamento *multi-tenant* (acesso a recurso de outro produtor).
`EntityNotFoundException`	`404 Not Found`	`Recurso Não Encontrado`	ID inexistente ou inconsistência hierárquica em rota aninhada (`/farms/{id}/...`).
`IllegalStateException`	`409 Conflict`	`Conflito de Regra de Negócio`	Violação das 3 travas *fail-fast*, MAC/Tag/Fazenda duplicada ou limpeza ativa no `DELETE`.
`Exception` (Fallback Geral)	`500 Internal Server Error`	`Erro Interno do Servidor`	Proteção contra vazamento de *stack traces* internos para o cliente externo.

Fonte: Elaborado pelos autores com base em `GlobalExceptionHandler.java` (2026).

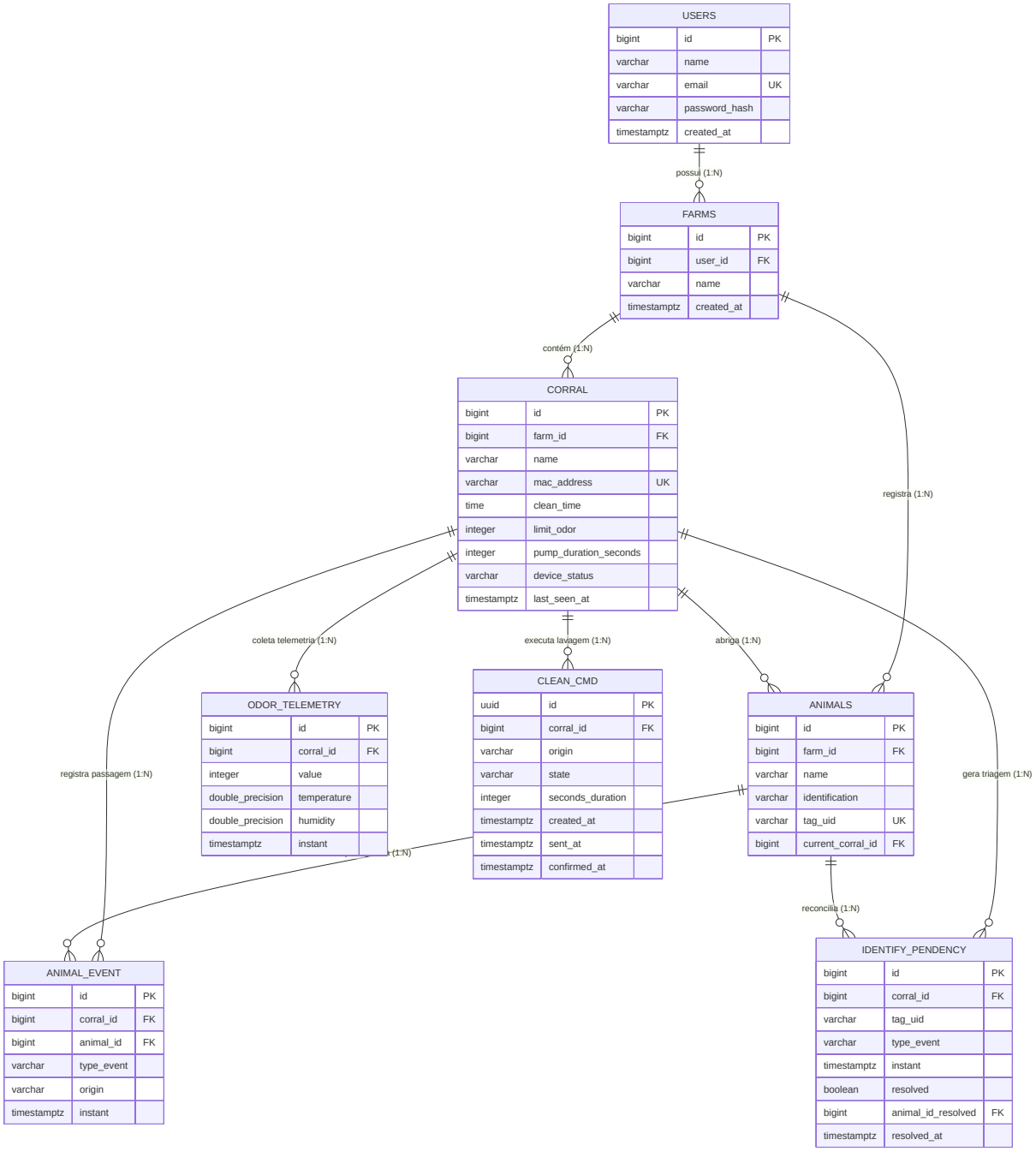
4.6 Modelagem Relacional no PostgreSQL 17, Migrações Flyway e Índices Parciais

O banco de dados relacional **PostgreSQL 17** é gerenciado por três migrações determinísticas do **Flyway 12.11.0** localizadas em `backend/src/main/resources/db/migration/`:

1. `V202610152058__create_initial_schema.sql`: Cria as 8 tabelas fundamentais do domínio (`users`, `farms`, `corral`, `animals`, `animal_event`, `odor_telemetry`, `clean_cmd`, `identify_pendency`), chaves estrangeiras com políticas de cascata (`ON DELETE CASCADE` e `ON DELETE SET NULL`), restrições de domínio (`CHECK`) e índices parciais;
2. `V202610300100__add_temperature_humidity_to_odor_telemetry.sql`: Evolui a tabela `odor_telemetry` adicionando as colunas `temperature DOUBLE PRECISION NULL` e `humidity DOUBLE PRECISION NULL` para armazenar os dados climáticos enviados pelo sensor DHT acoplado ao ESP32;
3. `V202610300101__allow_zero_duration_for_stop_clean_cmd.sql`: Atualiza a restrição `chk_clean_cmd_duration` na tabela `clean_cmd` para `CHECK (seconds_duration >= 0)`, permitindo persistir registros auditáveis de parada imediata da bomba (`stopClean`) com duração zero.

A Figura 6 apresenta o Diagrama Entidade-Relacionamento (DER) completo do banco de dados, e o Quadro 7 resume as regras de integridade física implementadas no SGBD. Para eliminar o problema clássico de `LazyInitializationException` em threads assíncronas do MQTT e evitar o problema de consultas N + 1, os repositórios JPA utilizam `@EntityGraph` (ex.: `CorralRepository.findByMacAddress` carrega `{"farm", "farm.user"}` em um único `JOIN` SQL).

Figura 6 — Diagrama Entidade-Relacionamento (DER Físico) do Banco de Dados PostgreSQL 17 Gerenciado pelo Flyway 12



Fonte: Elaborado pelos autores com base nos scripts de migração do Flyway (2026).

Quadro 7 — Dicionário de Tabelas, Restrições de Integridade (`CHECK`) e Índices Parciais no PostgreSQL 17

Tabela Relacional	Chave Primária	Restrições de Domínio (`CHECK` / `UNIQUE`)	Índices Especializados e Parciais
`users`	`id BIGINT IDENTITY`	`email UNIQUE NOT NULL`	PK B-Tree (`users_pkey`) + Unique (`users_email`
`farms`	`id BIGINT IDENTITY`	`user_id REFERENCES users(id) ON DELETE CASCADE`	`idx_farms_user_id ON farms(user_id)`
`corral`	`id BIGINT IDENTITY`	`mac_address ~ '^[0-9A-F]{12}$'` (Regex POSIX), `pump_duration_seconds > 0`, `device_status IN ('ONLINE','OFFLINE')`	`idx_corral_farm_id ON corral(farm_id)` + Unique `mac_address`
`animals`	`id BIGINT IDENTITY`	`tag_uid UNIQUE NOT NULL`, `current_corral_id REFERENCES corral(id) ON DELETE SET NULL`	`idx_animals_farm_id`, `idx_animals_current_corral`
`animal_event`	`id BIGINT IDENTITY`	`type_event IN ('ENTROU','SAIU')`, `origin IN ('TAG','MANUAL')`	`idx_animal_event_corral(corral_id, ins` `DESC)`, `idx_animal_event_animal`
`odor_telemetry`	`id BIGINT IDENTITY`	`value INTEGER NOT NULL`, `temperature DOUBLE PRECISION`, `humidity DOUBLE PRECISION`	`idx_odor_telemetry_corral_instant(corr` `instant DESC)`
`clean_cmd`	`id UUID`	`origin IN ('AUTOMATICA','MANUAL','EVENTO')`, `state IN ('PENDENTE','EM_ANDAMENTO','CONCLUIDO','CANCELADO')`, `seconds_duration >= 0`	**Índice Único Parcial:** `uq_clean_cmd_active_per_corral ON clean_cmd(corral_id) WHERE state IN ('PENDENTE','EM_ANDAMENTO')`
`identify_pendency`	`id BIGINT IDENTITY`	`type_event IN ('ENTROU','SAIU')`, `resolved BOOLEAN DEFAULT FALSE`	**Índice Parcial de Fila:** `idx_identify_pendency_open_corral ON identify_pendency(corral_id, instant D` `WHERE resolved = FALSE`

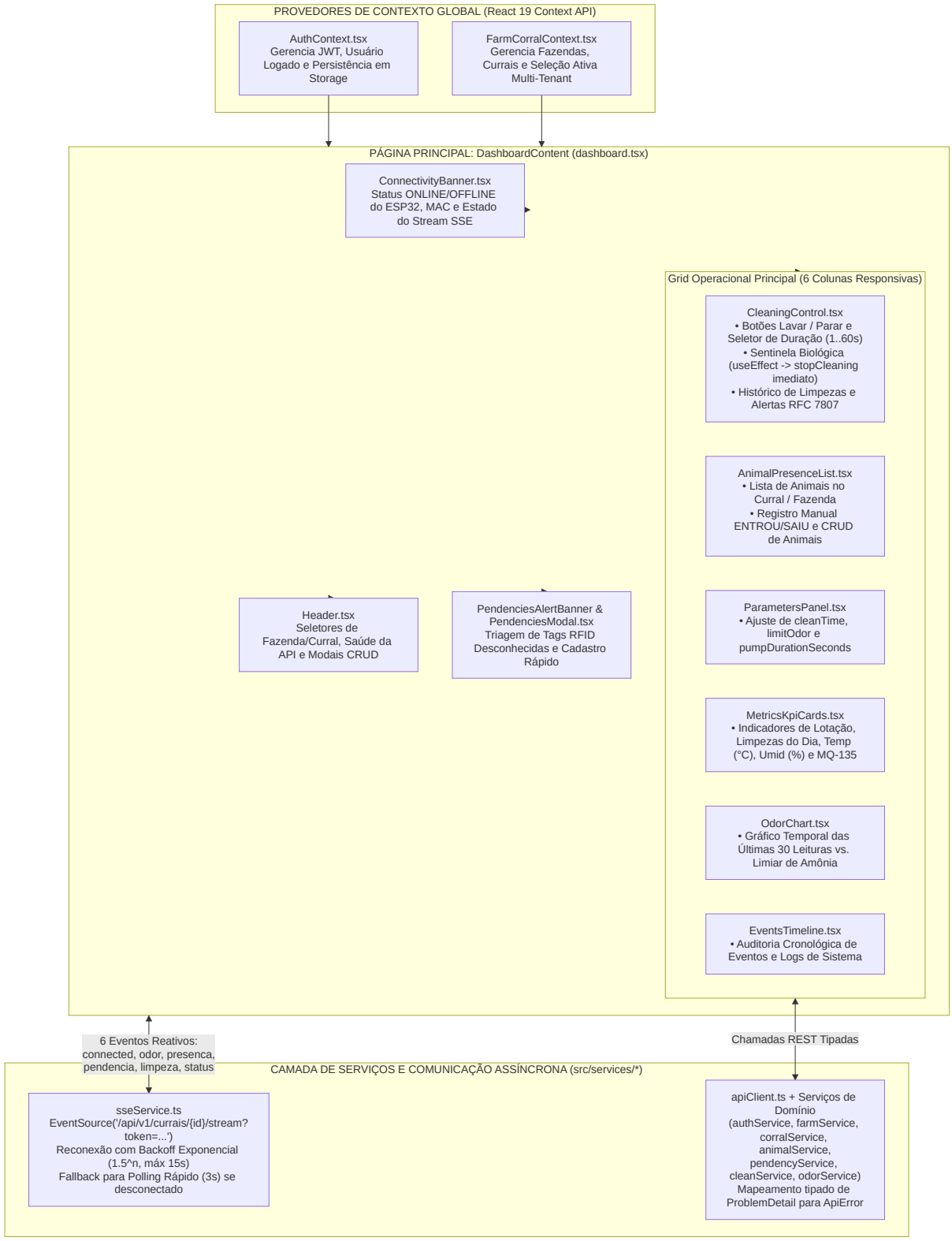
Fonte: Elaborado pelos autores com base nas migrações Flyway (2026).

5 ENGENHARIA DO FRONTEND REATIVO E STREAMING EM TEMPO REAL

5.1 Arquitetura Modular em React 19 e Gerenciamento de Contexto Global

A camada de apresentação foi construída como uma *Single-Page Application* (SPA) em **React 19.1.0** e **TypeScript 5.9.3**, estruturada em torno de dois provedores de estado global (`AuthContext` para gestão de sessão JWT e `FarmCorralContext` para seleção multi-fazenda e multi-curral) e treze subcomponentes modulares especializados localizados em `frontend/src/components/dashboard/`: `ConnectivityBanner`, `Header`, `CleaningControl`, `AnimalPresenceList`, `ParametersPanel`, `MetricsKpiCards`, `OdorChart`, `EventsTimeline`, `PendenciesModal`, `FarmModal`, `CorralModal`, `AuthDialog` e `ConfirmDeleteDialog`, conforme representado na Figura 9.

Figura 9 — Diagrama de Componentes da Arquitetura Reativa do Frontend (React 19, Context API e *Server-Sent Events*)



Fonte: Elaborado pelos autores com base no código-fonte de `frontend/src/` (2026).

5.2 Streaming Reativo via *Server-Sent Events* com *Backoff* Exponencial e *Fallback* de *Polling*

Para oferecer visibilidade instantânea sem sobrecarregar o servidor com requisições HTTP repetitivas, o serviço `CorralSseService` no backend mantém um mapa concorrente `ConcurrentHashMap<Long, List<SseEmitter>>` utilizando listas `CopyOnWriteArrayList<SseEmitter>` por curral. Sempre que ocorre uma mutação de estado (leitura de odor/temperatura/umidade, passagem

RFID, nova pendência ou resolução, início/conclusão/cancelamento de limpeza, ou mudança de status do ESP32), o evento é difundido (*multicast*) para todos os navegadores autenticados que monitoram aquele curral.

No cliente (`sseService.ts`), a função `subscribeToCorralStream` gerencia a conexão `EventSource` escutando 6 eventos nomeados (`connected`, `odor`, `presenca`, `pendencia`, `limpeza`, `status`) e aplica duas estratégias de alta disponibilidade:

1. **Reconexão Automática com *Backoff* Exponencial Limitado:** Em caso de queda de rede, o cliente agenda nova tentativa aplicando a progressão geométrica $T_{delay} = \min(T_{base} \times 1{,}5^{n},\ 15.000\ ms)$ por até 10 tentativas consecutivas, evitando tempestades de reconexão (*thundering herd*) no servidor;
2. **Comutação Adaptativa de *Polling* (`dashboard.tsx:608`):** Enquanto o túnel SSE encontra-se ativo (`isSseConnected == true`), o dashboard mantém uma sincronização silenciosa de reconciliação a cada **15 segundos** (`pollIntervalMs = 15000`); caso o stream SSE sofra desconexão, o intervalo de *polling* reduz-se automaticamente para **3 segundos** (`pollIntervalMs = 3000`), garantindo continuidade operacional ininterrupta.

5.3 Parada Biológica de Emergência em Camada Dupla (*Defense in Depth*) e Modo Simulado

Além da Trava Biológica prévia no backend, o frontend implementa uma **Sentinela Biológica Reativa em Tempo Real** tanto no orquestrador `dashboard.tsx` (e no *callback* `onPresenca` do SSE em `dashboard.tsx:373-381`) quanto no componente `CleaningControl.tsx`. Caso uma lavagem esteja em andamento (`washingRef.current == true`) e um animal adentre o curral naquele instante (detectado via evento SSE `"presenca"` ou atualização de estado), o `useEffect` sentinela interrompe imediatamente o cronômetro visual, exibe o alerta crítico `"PARADA DE SEGURANÇA: animal detectado no curral durante a lavagem! Bomba desligada."` e despacha assincronamente `cleanService.stopCleaning(corralId)` (`POST /api/v1/currais/{corralId}/limpeza/parar`), ordenando ao ESP32 que desarme imediatamente o relé físico no pino `GPIO 26`.

Adicionalmente, quando o usuário acessa a aplicação sem estar autenticado (`!isAuthenticated`), o sistema opera em **Modo de Demonstração Interativo Simulado**, permitindo testar localmente a lógica de lavagem, parada biológica de emergência, agendamento por horário e gráficos de sensores durante apresentações acadêmicas mesmo sem conectividade externa.

6 VERIFICAÇÃO, VALIDAÇÃO E RESULTADOS EXPERIMENTAIS

6.1 Metodologia de Verificação e Cobertura de Testes Automatizados (690 Testes)

Para comprovar empiricamente a corretude funcional, a segurança *multi-tenant* e a resiliência a falhas de rede e entradas malformadas, o projeto foi submetido a uma metodologia de verificação em múltiplos níveis: testes unitários isolados, testes de fatia web (`@WebMvcTest` / `MockMvc`), testes de integração com banco de dados PostgreSQL real, testes de componentes de interface (Vitest + React Testing Library + `jsdom`) e baterias de **testes adversariais (*Adversarial Challenge Tests*)** desenhados especificamente para tentar violar as regras de negócio, injetar payloads MQTT corrompidos e forçar condições de corrida.

Ao todo, o projeto contabiliza **690 testes automatizados** (345 no Backend e 345 no Frontend), com **0 falhas e 0 erros**.

6.2 Análise da Suíte de Testes do Backend (345 Testes em JUnit 5)

A execução da suíte de testes do backend via Maven Surefire (`./mvnw test`) abrange **23 classes de teste** e **345 casos de teste individuais** (incluindo classes aninhadas `@Nested`), todos aprovados com 100% de sucesso, conforme detalhado na Tabela 1 (formatada segundo as Normas de Apresentação Tabular do IBGE).

Tabela 1 — Consolidação Quantitativa da Suíte de Testes Automatizados do Backend (JUnit 5 / Surefire)

Classe de Teste (`com.usjt.a3.*`)	Escopo de Verificação Técnica	Testes Executados	Falhas / Erros	Tempo (s)
`A3ApplicationTests`	Fluxo E2E completo (PostgreSQL + HiveMQ + ACK ESP32/Wokwi + `stopClean`)	1	0	2,099
`adversarial.M1AdversarialChallengeTest`	Testes adversariais de autenticação, JWT inválido e limites de entrada	27	0	2,016
`adversarial.M2AdversarialChallengeTest`	Ataques IDOR *multi-tenant*, colisões de MAC/Tag e travas de exclusão	22	0	0,473
`adversarial.M3AdversarialChallengeTest`	Condições de contorno em parâmetros, pendências e travas *fail-fast*	20	0	0,291
`adversarial.M3IngestionAndMetricsChallengeTest`	Estresse de ingestão MQTT malformada e precisão das métricas diárias	21	0	0,376
`controller.AnimalControllerTest`	Rotas diretas e aninhadas de animais, presença e linha do tempo	20	0	0,192
`controller.CleanControllerTest`	Acionamento manual (`POST /limpeza`), parada (`POST /parar`) e histórico	9	0	0,109
`controller.CorralControllerTest`	CRUD de currais, `/parametros`, `/metricas`, `/stream` SSE e RFC 7807	24	0	0,283
`controller.FarmControllerTest`	CRUD de fazendas, unicidade de nome e bloqueio de exclusão ativa	9	0	0,103
`controller.LifecycleBoundaryAndRoutingEmpiricalTest`	Testes empíricos de fronteira de roteamento REST e códigos HTTP	28	0	0,483
`controller.PendencyControllerTest`	Listagem e resolução transacional de pendências RFID (`POST`/`PUT`)	11	0	0,139
`dto.request.CorralParametersValidationTest`	Bean Validation (`@Positive`, `@Max(60)`, `@PositiveOrZero`) em DTOs	20	0	0,024
`mqtt.entry.CommandRfidLeituraRealDbTest`	Integração real com banco PostgreSQL para leitura RFID e pendências	4	0	9,064
`mqtt.entry.CommandRfidLeituraTest`	Aliases de campos JSON, normalização de MAC/UID e tolerância a falhas	30	0	0,113
`mqtt.entry.MqttEntryRfidIntegrationTest`	Roteamento de canal Spring Integration MQTT até `CommandRfidLeitura`	1	0	0,060
`mqtt.entry.MqttTopicsTest`	Extração e validação de MAC Address e sufixos `EntryTopics`	4	0	0,001
`service.AnimalServiceTest`	Regra SSOT, transferência automática entre currais e validações manuais	32	0	0,096
`service.CleanServiceTest`	3 travas *fail-fast*, `AckService`, rollback `CANCELADO` e `stopClean`	6	0	0,158
`service.CorralServiceTest`	Regras de negócio de currais, *heartbeat* de 60s e agregação de métricas	22	0	0,025
`service.CorralSseServiceTest`	Subscrição `SseEmitter`, validação *multi-tenant* e remoção de inativos	3	0	0,003
`service.FarmServiceTest`	Isolamento de fazendas por produtor e proteção contra exclusão insegura	9	0	0,070
`service.IdentifyPendencyServiceTest`	Ciclo completo de criação e resolução de `IdentifyPendency` + auditoria	14	0	0,024
`service.LifecycleServiceBoundaryEmpiricalTest`	Fronteiras transacionais entre serviços de fazenda, curral e animais	8	0	0,059
Total Consolidado (23 Classes)	**Cobertura Integral do Backend Distribuído**	**345**	**0**	**16,061**

Fonte: Elaborado pelos autores a partir dos relatórios XML do Maven Surefire (`backend/target/surefire-reports/TEST-*.xml`) (2026).

6.3 Análise da Suíte de Testes do Frontend (345 Testes em Vitest e RTL)

No frontend, a execução da suíte automatizada via Vitest (`npm test`) totaliza **27 arquivos de teste** contendo **345 testes** (328 testes unitários, de componentes e adversariais executados em memória com 100% de aprovação em 11,48 segundos, acrescidos de 17 testes condicionais de *roundtrip* contra backend ao vivo em `liveBackendRoundtrip.test.ts`), além de 5 suítes de cenários *End-to-End* (`tests/e2e/tier1_features.test.ts` a `m3_adversarial_challenge.test.ts`), conforme consolidado na Tabela 2.

Tabela 2 — Consolidação Quantitativa da Suíte de Testes Automatizados do Frontend (Vitest / React Testing Library)

Categoria / Agrupamento de Arquivos de Teste (`frontend/tests/*`)	Arquivos	Casos de Teste	Aprovados	Ignorados (Live)	Falhas
Testes de Serviços e Clientes (`tests/services/*.test.ts`) (`apiClient`, `authService`, `farmService`, `corralService`, `animalService`, `pendencyService`, `cleanService`, `odorService`, `macUtils`, `sseService`, `empiricalChallenge`)	11	126	126	0	0
Testes de Componentes e Fluxos Reativos (`tests/components/*.test.tsx`) (`Dashboard`, `DashboardSse`, `AuthDialog`, `CleaningControl`, `ParametersPanel`, `AnimalPresenceList`, `HeaderAndModals`, `ConnectivityAndOdor`, `AutoDetectionAndEditFeatures`, `PendencyCreateAnimal`)	10	95	95	0	0
Testes Adversariais e de Estresse de UI (`tests/components/*Challenge*.test.tsx`) (`authDialogChallenge`, `cleaningTelemetryChallenge`, `empiricalChallengeUI`, `empiricalOdorAdversarialChallenge`, `m3Challenger2StressHarness`)	5	107	107	0	0
Testes Condicionais de Integração *Live Roundtrip* (`liveBackendRoundtrip.test.ts`)	1	17	—	17	0
Total Consolidado da Suíte Vitest	**27**	**345**	**328**	**17**	**0**

Fonte: Elaborado pelos autores a partir da execução do comando `vitest run` v3.2.7 (2026).

7 CONCLUSÃO E TRABALHOS FUTUROS

7.1 Considerações Finais

O presente relatório técnico documentou a concepção, a arquitetura, a implementação e a validação empírica do sistema **Curral Inteligente**, cumprindo integralmente os objetivos propostos para a Unidade Curricular Projeto A3 da Universidade São Judas Tadeu (USJT). A solução desenvolvida comprova que é tecnicamente viável aliar automação agropecuária de baixo custo baseada no microcontrolador ESP32 a práticas rigorosas de Engenharia de Software corporativa e Sistemas Distribuídos.

Os principais marcos técnicos alcançados pelo projeto incluem:

1. **Garantia Determinística do Bem-Estar Animal:** A combinação das três travas atômicas *fail-fast* no backend (`CleanService`) com a Sentinela Biológica Reativa no frontend (`CleaningControl` + `dashboard.tsx`) e o comando de parada imediata (`stopClean`) assegura proteção em múltiplas camadas contra acionamentos acidentais da bomba d'água na presença de bovinos;
2. **Eficiência e Escalabilidade de Recursos no Servidor:** O desacoplamento entre as transações de banco de dados (< 2 ms) e a janela de espera assíncrona do rádio IoT (até 10 s via `CompletableFuture` no `AckService`) elimina por completo o risco de *HikariCP Connection Starvation*, enquanto o roteamento MQTT em O(1) via padrão *Strategy* e o broadcast reativo via *Server-Sent Events* (`CorralSseService`) garantem baixa latência;
3. **Confiabilidade Elétrica e Autonomia na Borda:** A implementação de lógica *Active-LOW* com proteção contra *glitch* de inicialização no pino `GPIO 26` e o uso de temporização não-bloqueante via `millis()` nas duas variantes de firmware (`Iot.ino` e `IotV2.ino`) garantem que o hardware jamais alague o curral mesmo diante de queda total de conectividade Wi-Fi;
4. **Qualidade Comprovada por Testes Automatizados:** Os **690 testes automatizados** (345 no backend JUnit 5 e 345 no frontend Vitest) validam exaustivamente desde contratos REST e regras *multi-tenant* até cenários adversariais de falha de rede e concorrência.

7.2 Limitações Identificadas e Direções para Trabalhos Futuros

Como oportunidades de evolução para implantação industrial em larga escala, destacam-se quatro trabalhos futuros:

1. **Coordenação Distribuída de Agendamento (*ShedLock* / Redis):** Em cenários nos quais o backend Spring Boot seja escalado horizontalmente em múltiplas réplicas (*cluster* Kubernetes), recomenda-se integrar a biblioteca `ShedLock` ao `CleanSchedulerService` e migrar o estado volátil do `AckService` para Redis Pub/Sub, garantindo coordenação distribuída entre nós;
2. **Segurança Mútua em Camada de Transporte no Broker MQTT (mTLS):** Substituir a conexão MQTT em texto claro (porta 1883) por MQTT sobre TLS 1.3 (porta 8883) com certificados X.509 provisionados na memória *Flash* NVS do ESP32;
3. **Sensores Industriais de Amônia Eletroquímicos e Antenas RFID UHF:** Para currais comerciais de grande largura de porteira, substituir o leitor HF de curta distância MFRC522 (13,56 MHz) por leitores RFID UHF de longo alcance (ISO 18000-6C, 915 MHz), mantendo exatamente o mesmo contrato JSON no tópico `{prefixo}/{MAC}/rfid/leitura`;
4. **Modelos Preditivos de Conforto Térmico (ITU):** Utilizar as séries temporais de temperatura e umidade já armazenadas em `odor_telemetry` para calcular automaticamente o Índice de Temperatura e Umidade (ITU) e acionar sistemas de nebulização para mitigação de estresse calórico bovino.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023**: informação e documentação: referências: elaboração. 2. ed. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024**: informação e documentação: numeração progressiva das seções de um documento: apresentação. 2. ed. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027**: informação e documentação: sumário: apresentação. 2. ed. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028**: informação e documentação: resumo, resenha e recensão: apresentação. 2. ed. Rio de Janeiro: ABNT, 2021.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520**: informação e documentação: citações em documentos: apresentação. 2. ed. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10719**: informação e documentação: relatório técnico e/ou científico: apresentação. 4. ed. Rio de Janeiro: ABNT, 2015.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. 3. ed. Rio de Janeiro: ABNT, 2011.

ESPRESSIF SYSTEMS. **ESP32 Series Datasheet**: version 4.3. Shanghai: Espressif Systems, 2023.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projetos**: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Normas de apresentação tabular**. 3. ed. Rio de Janeiro: IBGE, 1993.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 20922:2016**: Information technology — Message Queuing Telemetry Transport (MQTT) v3.1.1. Geneva: ISO, 2016.

KLEINSCHMIDT, João Henrique *et al.* Internet of Things in agribusiness: architectures, protocols and challenges. **IEEE Internet of Things Journal**, New York, v. 6, n. 5, p. 8120-8135, out. 2019.

KLEPPMANN, Martin. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

MARTIN, Robert Cecil. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

NEWMAN, Sam. **Building microservices**: designing fine-grained systems. 2. ed. Sebastopol: O'Reilly Media, 2021.

NOTTINGHAM, Mark; WILDE, Erik; DALAL, Sanjay. **RFC 7807**: Problem Details for HTTP APIs. Fremont: IETF, mar. 2016. Disponível em: https://datatracker.ietf.org/doc/html/rfc7807. Acesso em: 26 out. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

TANENBAUM, Andrew Stuart; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2017.

APÊNDICE A — CATÁLOGO COMPLETO DE ENDPOINTS DA API REST

Quadro 8 — Catálogo Completo de Endpoints da API REST (`/api/v1`)

Módulo / Controller	Método HTTP	Rota Principal (`/api/v1/...`)	Rota Aninhada Equivalente	Status Sucesso	I
AuthController	POST	/auth/register	—	201 Created	R t
AuthController	POST	/auth/login	—	200 OK	A e
FarmController	POST	/farms	—	201 Created	C p a
FarmController	GET	/farms	—	200 OK	L p
FarmController	PUT	/farms/{farmId}	—	200 OK	A f u
FarmController	DELETE	/farms/{farmId}	—	204 No Content	E c l
CorralController	POST	/farms/{farmId}/currais	—	201 Created	C a h
CorralController	GET	/farms/{farmId}/currais	—	200 OK	L c (
CorralController	GET	/currais/{corralId}	/farms/{farmId}/currais/{corralId}	200 OK	C c c
CorralController	PUT	/currais/{corralId}	/farms/{farmId}/currais/{corralId}	200 OK	A e
CorralController	DELETE	/currais/{corralId}	/farms/{farmId}/currais/{corralId}	204 No Content	E c l
CorralController	PUT	/currais/{corralId}/parametros	/farms/{farmId}/currais/{corralId}/parametros	200 OK	A l p (
CorralController	GET	/currais/{corralId}/metricas	/farms/{farmId}/currais/{corralId}/metricas	200 OK	E c c
CorralController	GET	/currais/{corralId}/stream	/farms/{farmId}/currais/{corralId}/stream	200 OK (SSE)	S t (
AnimalController	POST	/farms/{farmId}/animais	—	201 Created	C E
AnimalController	GET	/farms/{farmId}/animais	—	200 OK	L c
AnimalController	GET	/animais/{animalId}	/farms/{farmId}/animais/{animalId}	200 OK	C a
AnimalController	PUT	/animais/{animalId}	/farms/{farmId}/animais/{animalId}	200 OK	A T
AnimalController	DELETE	/animais/{animalId}	/farms/{farmId}/animais/{animalId}	204 No Content	E c E
AnimalController	POST	/currais/{corralId}/presenca	/farms/{farmId}/currais/{corralId}/presenca	201 Created	R r e
AnimalController	GET	/currais/{corralId}/eventos	/farms/{farmId}/currais/{corralId}/eventos	200 OK	L a c
PendencyController	GET	/currais/{corralId}/pendencias	/farms/{farmId}/currais/{corralId}/pendencias	200 OK	L p (
PendencyController	POST/PUT	/pendencias/{pendencyId}/resolver	—	200 OK	E E a
CleanController	POST	/currais/{corralId}/limpeza	—	200 OK	A v e

Módulo / Controller	Método HTTP	Rota Principal (`/api/v1/...`)	Rota Aninhada Equivalente	Status Sucesso	I
`CleanController`	`POST`	`/currais/{corralId}/limpeza/parar`	`/currais/{corralId}/parar`	`200 OK`	I a F
`CleanController`	`GET`	`/currais/{corralId}/limpezas`	`/currais/{corralId}/limpeza`	`200 OK`	I c c
`OdorController`	`GET`	`/currais/{corralId}/odor`	—	`200 OK`	F l 1 u

Fonte: Elaborado pelos autores com base nos controladores do pacote `com.usjt.a3.controller` (2026).

APÊNDICE B — ESQUEMA DDL CONSOLIDADO DO BANCO DE DADOS (FLYWAY)

```
-- ============================================================================
-- MIGRAÇÃO 1: V202610152058__create_initial_schema.sql
-- ============================================================================
CREATE TABLE users (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE farms (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_farms_user_id ON farms(user_id);

CREATE TABLE corral (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    farm_id BIGINT NOT NULL REFERENCES farms(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    mac_address VARCHAR(12) NOT NULL UNIQUE,
    clean_time TIME NULL,
    limit_odor INTEGER NOT NULL DEFAULT 400,
    pump_duration_seconds INTEGER NOT NULL DEFAULT 30,
    device_status VARCHAR(15) NOT NULL DEFAULT 'OFFLINE',
    last_seen_at TIMESTAMPTZ NULL,
    CONSTRAINT chk_corral_mac_address CHECK (mac_address ~ '^[0-9A-F]{12}$'),
    CONSTRAINT chk_corral_pump_duration CHECK (pump_duration_seconds > 0),
    CONSTRAINT chk_corral_device_status CHECK (device_status IN ('ONLINE', 'OFFLINE'))
);
CREATE INDEX idx_corral_farm_id ON corral(farm_id);

CREATE TABLE animals (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    farm_id BIGINT NOT NULL REFERENCES farms(id) ON DELETE CASCADE,
    name VARCHAR(50) NOT NULL,
    identification VARCHAR(30) NULL,
    tag_uid VARCHAR(32) NOT NULL UNIQUE,
    current_corral_id BIGINT NULL REFERENCES corral(id) ON DELETE SET NULL
);
CREATE INDEX idx_animals_farm_id ON animals(farm_id);
CREATE INDEX idx_animals_current_corral ON animals(current_corral_id);

CREATE TABLE animal_event (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    corral_id BIGINT NOT NULL REFERENCES corral(id) ON DELETE CASCADE,
    animal_id BIGINT NOT NULL REFERENCES animals(id) ON DELETE CASCADE,
    type_event VARCHAR(10) NOT NULL,
    origin VARCHAR(10) NOT NULL DEFAULT 'TAG',
    instant TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_animal_event_type CHECK (type_event IN ('ENTROU', 'SAIU')),
    CONSTRAINT chk_animal_event_origin CHECK (origin IN ('TAG', 'MANUAL'))
);
CREATE INDEX idx_animal_event_corral ON animal_event(corral_id, instant DESC);
CREATE INDEX idx_animal_event_animal ON animal_event(animal_id, instant DESC);

CREATE TABLE odor_telemetry (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    corral_id BIGINT NOT NULL REFERENCES corral(id) ON DELETE CASCADE,
    value INTEGER NOT NULL,
    instant TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_odor_telemetry_corral_instant ON odor_telemetry(corral_id, instant DESC);

CREATE TABLE clean_cmd (
    id UUID PRIMARY KEY,
    corral_id BIGINT NOT NULL REFERENCES corral(id) ON DELETE CASCADE,
    origin VARCHAR(20) NOT NULL,
    state VARCHAR(25) NOT NULL,
    seconds_duration INTEGER NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    sent_at TIMESTAMPTZ NULL,
    confirmed_at TIMESTAMPTZ NULL,
    CONSTRAINT chk_clean_cmd_origin CHECK (origin IN ('AUTOMATICA', 'MANUAL', 'EVENTO')),
    CONSTRAINT chk_clean_cmd_state CHECK (state IN ('PENDENTE', 'EM_ANDAMENTO', 'CONCLUIDO', 'CANCELADO')),
    CONSTRAINT chk_clean_cmd_duration CHECK (seconds_duration > 0)
);
CREATE INDEX idx_clean_cmd_corral_state ON clean_cmd(corral_id, state);
CREATE UNIQUE INDEX uq_clean_cmd_active_per_corral
    ON clean_cmd(corral_id)
    WHERE state IN ('PENDENTE', 'EM_ANDAMENTO');

CREATE TABLE identify_pendency (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    corral_id BIGINT NOT NULL REFERENCES corral(id) ON DELETE CASCADE,
    tag_uid VARCHAR(32) NULL,
    type_event VARCHAR(10) NOT NULL DEFAULT 'SAIU',
    instant TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved BOOLEAN NOT NULL DEFAULT FALSE,
    animal_id_resolved BIGINT NULL REFERENCES animals(id) ON DELETE SET NULL,
    resolved_at TIMESTAMPTZ NULL,
    CONSTRAINT chk_identify_pendency_type_event CHECK (type_event IN ('ENTROU', 'SAIU'))
);
CREATE INDEX idx_identify_pendency_open_corral
    ON identify_pendency(corral_id, instant DESC)
    WHERE resolved = FALSE;

-- ================================================================================
-- MIGRAÇÃO 2: V202610300100__add_temperature_humidity_to_odor_telemetry.sql
-- ================================================================================
ALTER TABLE odor_telemetry ADD COLUMN IF NOT EXISTS temperature DOUBLE PRECISION NULL;
ALTER TABLE odor_telemetry ADD COLUMN IF NOT EXISTS humidity DOUBLE PRECISION NULL;

-- ================================================================================
-- MIGRAÇÃO 3: V202610300101__allow_zero_duration_for_stop_clean_cmd.sql
-- ================================================================================
ALTER TABLE clean_cmd DROP CONSTRAINT IF EXISTS chk_clean_cmd_duration;
ALTER TABLE clean_cmd ADD CONSTRAINT chk_clean_cmd_duration CHECK (seconds_duration >= 0);
```